



Expert Insights
PERSPECTIVE ON A TIMELY POLICY ISSUE

ALLISON BERKE, KYLE A. KILIAN, GRAHAM GRIFFIN, JAY ATANDA, ENRIQUE VAZQUEZ

Verifiable Audit Trails for AI-Enabled Biological Design Tools

A Proposal for Verifiable Biodesign Logging

For more information on this publication, visit www.rand.org/t/PEA4613-1.

About RAND

RAND is a research organization that develops solutions to public policy challenges to help make communities throughout the world safer and more secure, healthier and more prosperous. RAND is nonprofit, nonpartisan, and committed to the public interest. To learn more about RAND, visit www.rand.org.

Research Integrity

Our research integrity is grounded in RAND's core values of quality and objectivity. Rigorous quality assurance procedures, conflict of interest screening, and transparency in funding ensure that every study is objective and nonpartisan. Learn more at www.rand.org/integrity.

RAND's publications do not necessarily reflect the opinions of its research clients and sponsors.

Published by the RAND Corporation, Santa Monica, Calif.

© 2026 RAND Corporation

RAND® is a registered trademark.

Limited Print and Electronic Distribution Rights

This publication and trademark(s) contained herein are protected by law. This representation of RAND intellectual property is provided for noncommercial use only. Unauthorized posting of this publication online is prohibited; linking directly to its webpage on rand.org is encouraged. Permission is required from RAND to reproduce, or reuse in another form, any of its research products for commercial purposes. For information on reprint and reuse permissions, visit www.rand.org/about/publishing/permissions.

About This Paper

Unregulated access to artificial intelligence (AI)–accelerated protein-design platforms creates a verification gap: Synthesis providers and regulators cannot easily determine whether an uploaded sequence was derived using safe design objectives or tailored for harmful properties. Traditional documentation (PDF reports, notebooks) is mutable and tool-specific, leaving biosecurity stakeholders without a trustworthy chain of custody. The absence of a standard, tamper-evident audit trail undermines global efforts to screen DNA orders.

Through conversations with stakeholders including AI-enabled biological design (AI-bio) tool developers, AI-bio tool users, cryptography experts, and biosecurity experts, we developed a proposal for an extensible verifiable audit trail framework that can be used with any AI-bio tool and can be implemented in any computing environment that includes a trusted hardware enclave.

Center on AI, Security, and Technology

RAND Global and Emerging Risks is a division of RAND that delivers rigorous and objective public policy research on the most consequential challenges to civilization and global security. This work was undertaken by the division’s Center on AI, Security, and Technology, which aims to examine the opportunities and risks of rapid technological change, focusing on artificial intelligence, security, and biotechnology. For more information, contact cast@rand.org.

Funding

This effort was independently initiated and conducted within the Center on AI, Security, and Technology using income from operations and gifts and grants from philanthropic supporters. A complete list of donors and funders is available at www.rand.org/CAST. RAND clients, donors, and grantors have no influence over research findings or recommendations.

Acknowledgments

We first thank our reviewers, Chad Heitzenrater and Greg Butchello, for their insightful comments and help improving this work. We thank Sella Nevo and Henry Willis for providing support and research direction as part of the Meselson Center leadership team. AI (RAND Chat, an internal large language model) was used to generate Figure 1 from a text description, to identify sections in initial drafts that were unclear or jargon heavy, to suggest sections that could be reworded or rearranged, and to provide suggestions for additional explanation or detail to include to clarify the paper. All AI-generated suggestions were assessed, reworded, and integrated into the text by the authors if the suggestions were deemed valuable.

Summary

Advances in generative and predictive protein-design tools, including AlphaFold, RoseTTAFold, ProteinMPNN, and diffusion models, have lowered the barrier to creating novel biological materials (Jumper et al., 2021a). While these capabilities promise breakthroughs in drug discovery, they also raise governance challenges: How can regulators, funders, and collaborators verify what was designed, how, and by whom, especially when the designer cannot be fully trusted?

In this paper, we propose a cross-tool, tamper-evident audit-trail architecture that couples structured provenance records with hardware- or service-bound cryptographic signing. The solution builds on emerging open standards, such as the World Wide Web Consortium (W3C) PROV Ontology (PROV-O), and integrity concepts as implemented in such frameworks as in-toto and Sigstore and supported by trusted execution environments (TEEs) and remote notary services. Implemented correctly, the framework would enable the following (Lebo, Sahoo, and McGuinness, 2013):

- reproducibility for any third party to rerun a workflow and obtain identical results
- nonrepudiation, because designers cannot later deny responsibility
- tamper evidence of any post hoc modification to the log or outputs
- versatility, because the same schema applies across existing and future design platforms.

Issue

Increased access to and performance of biomolecule design platforms create a verification challenge. Synthesis providers and regulators cannot easily determine whether an uploaded sequence was derived using safe design objectives or tailored for harmful properties. Traditional documentation is mutable and tool-specific, leaving biosecurity stakeholders without a trustworthy chain of custody. The absence of a standard, tamper-evident audit trail undermines global efforts to screen DNA orders. This paper is intended for gene synthesis providers, such as those in the International Gene Synthesis Consortium (IGSC) and biotechnology regulatory agencies, that could be considering ways to mitigate risks from biodesign tools. It would also benefit anyone interested in developing audit software who would like to know key concerns.

Approach

We developed a proposal for an extensible verifiable audit-trail framework that can be used with any artificial intelligence (AI)-enabled biological design (AI-bio) tool and that can be implemented in any computing environment that includes a trusted hardware enclave. We made improvements based on conversations with stakeholders, including AI-bio tool developers, AI-bio tool users, cryptography experts, and biosecurity experts.

Conclusions

Our analysis of the verification gap in AI-driven protein design and of the technical and governance landscape surrounding it yielded the following conclusions:

- **The verification gap is real and growing.** Synthesis providers and regulators lack reliable means to determine whether a submitted sequence was generated using capability-amplifying AI tools, what opti-

mization objectives guided its design, or whether the designer’s stated description of the sequence accurately characterizes the design process. This gap exists not because the information is inherently unrecoverable, but because no systematic mechanism exists to capture and preserve it at the point of design.

- **The technical components required to close this gap exist and are mature.** Hash chaining, hardware-bound attestation, and append-only transparency logging are individually well established in adjacent high-assurance domains, including software supply chain security, certificate transparency, and financial audit systems. The novel contribution of this paper is the proposed composition of these components into a unified protocol tailored to the specific threat model of AI-driven biodesign. The feasibility of this composition is supported by the existence of operational analogs at comparable data volumes in those adjacent fields.
- **The primary barriers to deployment are not technical.** We identify three nontechnical barriers as more significant than any remaining technical challenge: the absence of governance structures to operate shared infrastructure, such as transparency logs and notary services; the distributional burden that an audit requirement would impose unevenly across the research community; and the gap between the existence of an audit record and the existence of institutional capacity to review and act on it. Each of these barriers requires policy and organizational action that is in addition to the technical protocol.
- **The assurance level of the system is bounded by its deployment configuration.** The protocol provides strong cryptographic guarantees about log integrity once data have been captured but does not resolve the question of how tool outputs are securely captured in the first place. The weakest deployment configuration, in which the mechanism of recording tool outputs wraps a tool as a subprocess, provides meaningful protection against casual falsification but would not withstand a technically sophisticated actor with full control of their own machine. Stronger configurations, including TEE co-execution, provide higher assurance but, as of this writing in 2026, are not compatible with most existing biodesign software. Any policy relying on the audit trail should specify which configuration is required for a given level of risk.
- **Simpler trust architectures warrant consideration alongside the proposed design.** Our analysis of the threat model suggests that a federated or centralized trust architecture, such as one built on existing institutional identity management infrastructure, could satisfy the core security requirements in many deployment contexts at substantially lower complexity than the fully decentralized baseline proposed here. The decentralized design is appropriate as a baseline for early deployment before governance structures exist but should not be treated as the permanent target architecture.

Recommendations

The conclusions above support the following recommendations, each of which is directed at a specific actor and grounded in the analysis:

- *Biosecurity policymakers and funders should commission a governance design process before investing further in technical development.* The analysis identifies governance gaps concerning who operates shared infrastructure, how disputes are adjudicated, and how the system is prevented from becoming a surveillance tool as binding constraints on deployment. Technical development that proceeds without resolving these questions risks producing a system that is cryptographically sound but institutionally undeployable. A structured process involving synthesis providers, research institutions, legal experts, and civil society representatives should precede significant investment in production infrastructure.
- *Synthesis provider consortia, including IGSC, should evaluate whether audit-trail submission could be incorporated into existing screening workflows as a voluntary pilot.* The analysis shows that the protocol is

compatible with existing order-review pipelines and that the verification steps it requires are analogous to screening that is already performed. A voluntary pilot with willing providers and a defined cohort of research institutions would generate empirical evidence about operational burden, false-positive rates in policy-gated notarization, and researcher compliance behavior—evidence that is absent in the existing infrastructure and that would be required to support any mandatory requirement.

- *Regulatory agencies with jurisdiction over dual-use research, including National Institutes of Health and potentially export control authorities, should assess whether the protocol's nonrepudiation properties create a sufficient evidentiary basis to extend existing dual-use research of concern frameworks to in silico protein design.* The analysis shows that a verified audit log materially changes the information available to regulators at the point of screening. However, the legal and regulatory implications of this change, including questions of liability, evidentiary standards, and jurisdictional reach over digital biological artifacts, have not been analyzed for this paper and require dedicated legal and policy analysis before any regulatory action is taken.
- *Software developers and AI-bio tool creators should engage early with the protocol specification to identify integration constraints.* The analysis identifies tool-level integration, specifically the secure capture of tool outputs by the recorder, as the point of highest practical uncertainty in the protocol. Developers of widely used platforms, such as AlphaFold, RoseTTAFold, and ProteinMPNN, are best positioned to assess whether TEE co-execution or kernel-level monitoring is feasible within their architectures and to identify the integration path of least resistance. This engagement should occur before a reference implementation is finalized, not after.
- *Any deployment of the protocol should be accompanied by an explicit specification of the assurance level in use and a corresponding statement of the residual risks.* Our analysis shows that the protocol's security properties vary significantly across deployment configurations and that the weakest configuration, though practically accessible, provides guarantees that may not be adequate for high-risk design categories. Policymakers, synthesis providers, and researchers relying on audit bundles should understand what configuration produced a given bundle and what the limits of the resulting assurance are. A tiered assurance schema should be developed, and standardizing the communication of assurance levels should be a priority in any pilot deployment.

Contents

About This Paper	iii
Summary	v
Figure and Tables	xi
CHAPTER 1	
Introduction	1
CHAPTER 2	
The Verification Gap in Protein Design	3
Information Life Cycle and Trust Assumptions	3
Attacker Model and Security Goals	4
CHAPTER 3	
Requirements and Design of an Audit System.....	7
Parties and Roles	7
Security Properties and Their Justifications	8
Protocol Phases.....	8
CHAPTER 4	
Trust Architecture and Deployment Configurations	13
Verification Workflow	14
Deployment Configuration 1: Hardware-Bound Local Signing.....	14
Deployment Configuration 2: Remote Notary Signing.....	17
Deployment Configuration 3: Offline and Air-Gapped Verification.....	18
Alternative Trust Architectures and Design Trade-Offs	20
CHAPTER 5	
Cross-Tool Generalizability and Common Workflows	23
What the Protocol Requires of a Tool Environment	23
What the Protocol Does Not Require	24
What Tool Owners Should Do	25
Cross-Tool Workflows.....	25
CHAPTER 6	
Policy Considerations and Conclusion.....	27
Policy Implications of Verifiable Audit Trails.....	27
Recommendations	28
Organizational Roles and Actionable Steps	29
Challenges.....	30
APPENDIX	
Commonly Used Biodesign Tools and Typical Workflows.....	33
Abbreviations.....	37
Glossary	39

References	43
About the Authors	47

Figure and Tables

Figure

4.1.	Automated Verification Workflow for Synthesis Providers	15
------	---	----

Tables

2.1.	Types of Information Logged in an Audit-Trail Document	4
4.1.	Roles, Actors, and Requirements for Implementing Verifiable Audit Trails	14
4.2.	Mapping of Each Verification Workflow Step to the Verifiable Audit-Trail Architecture	16
4.3.	Example of Software Audit Logs Already in Use.....	17
4.4.	Existing Services That Enable Remote Signing	18

Introduction

Over the past five years, machine learning–driven protein design has progressed from academic demonstrations to point-and-click commercial services available to students and hobbyists. AlphaFold’s public release in 2021 triggered a wave of derivative prediction and generative models (Baek et al., 2021; Dauparas et al., 2022; Jumper et al., 2021a; Lu et al., 2026). Diffusion-based backbones, language-model sequence generators, and automated *in silico* screening are now standard components of cloud biofoundries.

While these innovations accelerate legitimate research by shortening the path from concept to therapeutic lead, they also reduce the barriers to engineering potentially harmful proteins. The same concepts that could accelerate novel drug discovery also accelerate equally novel toxins, immune-evasive viral capsids, or catalytic domains that can amplify existing threats. In 2022, the World Health Organization and the National Science Advisory Board for Biosecurity both flagged artificial intelligence (AI)–enabled biological design (AI-bio) as an emerging dual-use concern (National Science Advisory Board for Biosecurity, 2023; World Health Organization, 2022).

A central biosecurity challenge is verifiable provenance prior to synthesis and shipping of genetic material (Crawford et al., 2024). Absent an immutable record, investigators can falsely claim that a design is benign, and reviewers cannot independently reproduce or audit the workflow. Even when required to present logs, such as notebooks, adversaries can retrospectively doctor records to discard incriminating steps. Laboratory documentation insufficiently satisfies the evidentiary standards for export-control determinations and the Biological Weapons Convention confidence-building measures. In addition, it can be challenging to verify customer authenticity if addresses and identities are able to be spoofed.

Therefore, in this paper, we propose and examine the feasibility of a cross-platform audit-trail architecture that treats every design run as a cryptographically sealed “flight recorder.” By binding each event to a hash chain, signing the root hash with a hardware-attested key outside the user’s control, and anchoring the signature in a public transparency log, regulators gain a tamper-evident ledger. This ledger can then be verified with commodity open-source tools, independent of the designer’s honesty or the specific software used.

The remainder of this paper proceeds as follows: Chapter 2 characterizes the verification gap in greater detail, examining current logging practices, the information that is and is not captured by existing tools, and the threat model that the protocol is designed to address. Chapters 3 through 5 then describe the protocol itself—covering its logical design and security properties, trust architecture, deployment configurations, and cross-tool generalizability. Chapter 6 turns to the policy implications and the governance, legal, and organizational challenges to be addressed alongside the technical work for the protocol to achieve its intended purpose.

The Verification Gap in Protein Design

Regulators, scientific journals, and security practitioners require provenance to assess dual-use risk and reproducibility (Chhetri et al., 2025). Our conversations with users of AI-bio tools and analysis of commonly used tools highlighted three pain points: inconsistent logging, poor cross-tool semantics, and user-controlled signing. Current tools produce logs intended for user benefit to iterate on experiments rather than to provide a comprehensive receipt for third parties. As a result, some tools may produce either proprietary log files or none at all. Relatedly, the diversity of tools leads to an equal diversity of data that are not easily comparable across tools. While the input(s) and final output(s) will typically be gene sequence files and protein structures, other metadata, such as scoring or model parameters, may be useful but can vary in their data type or inclusion. This variety can make it burdensome to parse through to understand the experimental process. Another overarching challenge is the extent of user control of files. A bad actor can edit any produced file to conceal evidence of malicious intent.

Information Life Cycle and Trust Assumptions

The information under consideration flows through three stages, each with distinct vulnerability surfaces. In the *design stage*, a researcher uses one or more AI-bio tools on a local workstation or cloud instance to generate a candidate sequence. The primary risk here is that log entries are suppressed, modified, or generated retroactively by a designer with full control of their own machine. In the *submission stage*, the resulting sequence and an accompanying audit bundle are transmitted to a synthesis provider for screening. The risk here is that the bundle is substituted or stripped in transit. In the *verification stage*, a synthesis provider or regulator independently checks the bundle's integrity and authenticity. The risk here is that a forged or backdated signature is accepted as valid. The architecture addresses each stage in turn: Hardware-bound signing inside a trusted execution environment (TEE) removes the designer's ability to forge or suppress log entries at the design stage, because the private key is never exposed to the host operating system; the cryptographic hash chain and in-toto attestation detect any modification during submission; and the transparency log provides an independent, append-only time stamp that makes backdating detectable at verification. The key trust assumption underlying the entire schema is that the TEE manufacturer and the transparency log operator are not colluding with the attacker, an assumption that is consistent with the threat model above, which focuses on individual designers rather than nation state-level adversaries capable of compromising hardware supply chains.

Table 2.1 summarizes our assessment of the primary loggable categories of data and why each is vital for risk assessment. To close the aforementioned gaps, an audit recorder can capture a rich, machine-readable footprint of every design session. The categories listed in Table 2.1 are available to most modern protein-design tools.

Identity and credential fields are strings referencing authenticated identifiers, such as Open Researcher and Contributor ID (ORCID) researcher IDs or institutional certificates; sequence fields are FASTA-format

TABLE 2.1
Types of Information Logged in an Audit-Trail Document

Category	Representative Fields	Biosecurity Value
User and session context	user_id (public-key thumbprint); institutional Open Identification Connect (OIDC); purpose tag (e.g., “therapeutic lead” versus “biochemistry class”); session_uuid; login internet protocol (IP); Geographical Internet Protocol (GeoIP) coarse location	Enable attribution and insider-threat deterrence
Time stamps	Start and stop times for each pipeline stage; monotonic clock skew record	Create event chronologies for forensic reconstruction
Input artifacts	Uploaded FAST-All (FASTA) or GenBank files; template Protein Data Bank (PDB) IDs; conditioning prompts; reference multiple sequence alignments (MSAs); secure hash algorithm (SHA) 256-bit (SHA-256) of each	Show whether restricted sequences were supplied
Design parameters	Hyperparameters (num_recycles, temperature, cfg_scale); objective functions (e.g., Rosetta score weights, diffusion loss terms); random seeds	Detect covert optimization for toxicity or immune evasion
External data sources	Exact database snapshot digests (e.g., UniProt release version, National Center for Biotechnology Information [NCBI] nonredundant [NR] 2025-05-01); AI model checkpoints	Reveal the use of disallowed or outdated data
Environment snapshots	Container image digest; Python or Compute Unified Device Architecture (CUDA) versions; graphics processing unit (GPU) model; kernel hash; software bill of materials (SBOM) for dependencies	Enable deterministic re-execution
Intermediate outputs	MSA depth metrics; validation scores; filtering thresholds; troubleshooting logs	Surface suspicious pruning of red-flag variants
Final results	Designed amino acid or nucleotide sequences; structural coordinates; in silico quality control reports	Constitute a core object for downstream synthesis screen
Hash-linking metadata	sha_prev hash chain pointer; Merkle leaf index; Rekord entry ID	Facilitate cryptographic immutability across records

text strings or cryptographic hashes of such strings in which the full sequence is stored separately; parameter fields are key-value pairs with typed values (numeric for thresholds and scores, enumerated strings for model names and objective functions); and time stamp fields conform to ISO 8601. This typing ensures that log fields are machine parsable for automated screening, and specific fields can be programmatically extracted and evaluated without manual inspection of an unstructured document.

Capturing this breadth of information ensures that regulators and synthesis providers can reproduce a result and evaluate the intent and risk posture underlying design choices, e.g., whether an optimizer was deliberately tuned to maximize a feature that could be harmful.

The risk of intentional misuse can be substantially reduced by answering the three following critical biosecurity questions: Who initiated the design and under what organizational credentials? What biological sequences, templates, and parameters were used or produced? When did each operation occur, relative to other activities on the same infrastructure? These can be answered through the aggregation and accurate logging of data generated during biodesign tool use (Tani, 2025).

Attacker Model and Security Goals

The primary attacker in this model is a designer, or someone acting on their behalf, who wishes to obscure the provenance of a biological sequence after the fact. This could involve claiming that a harmful design was generated by a benign, off-the-shelf tool rather than a capability-amplifying one; altering logged parameters

(such as an optimization objective) to conceal that a sequence was explicitly optimized for toxicity or immune evasion; or denying that a design run occurred at all. A secondary attacker class consists of external adversaries, such as a malicious insider at a synthesis provider or a third-party attacker who has compromised a designer's workstation, who might attempt to inject or delete log entries in transit or at rest. The architecture does not attempt to prevent a designer from conducting a harmful design run in the first place; rather, it ensures that if a run occurred, a tamper-evident record of it exists that the designer cannot plausibly disavow. Reproducibility and attribution are therefore key security properties, not merely scientific ones: They close the specific gap in which a designer could falsely characterize a sequence to a synthesis provider or regulator.

Requirements and Design of an Audit System

This chapter describes the verifiable audit trail (VAT) protocol at the level of its logical steps and security properties, independent of any specific implementation. The goal is to specify what each party must do, what artifacts are produced, and what security guarantees result, in a way that allows implementers to select appropriate technologies while ensuring that the essential properties of the schema are preserved.

The protocol described in this chapter draws on concepts from cryptography and computer hardware security that may be unfamiliar to readers without a technical background. Four concepts are foundational to understanding how the system works:

- A *hash function* is a mathematical operation that takes any piece of data, such as a file, a log entry, or a sequence, and produces a short, fixed-length fingerprint. The fingerprint is effectively unique: Any change to the original data, even a single character, produces a completely different fingerprint. This property is what makes tampering detectable.
- A *hash chain* is a sequence of records in which each record includes the fingerprint of the record before it; altering any record breaks all the fingerprints that follow, making the alteration immediately apparent.
- *Attestation* is a cryptographically verifiable claim that a specific, known piece of software is running in a specific environment.
- A *TEE* is an isolated region within a computer processor that is protected by hardware, such that even the owner of the device cannot read or modify what runs inside it.

Together, these four building blocks—hash functions, hash chains, attestation, and TEEs—allow the audit trail to be tamper evident, independently verifiable, and resistant to falsification by the designer. Readers seeking definitions of additional technical terms used in this chapter and in Chapters 4 and 5 should consult the glossary.

Parties and Roles

The protocol involves four parties. The *designer* is the individual or organization that uses AI-bio tools to generate a biological sequence. The *recorder* is a software process, operating under a hardware-enforced trust boundary on the designer's behalf, that observes and logs each step of a design run. The *notary* is an independent service that receives the completed log, validates it against defined policies, and issues a signed attestation. The *verifier* is any downstream party, such as a synthesis provider or regulator, that receives a design submission and wishes to confirm its provenance. These roles are not necessarily filled by different organizations; a synthesis provider could operate both a notary and a verifier. What matters is that each role carries distinct trust assumptions, described in the “Protocol Phases” section later in this chapter.

Security Properties and Their Justifications

Properties of security—completeness, immutability, usability, scalability, extensibility, and neutral trust anchor—can be mapped to the protocol steps. Completeness is a logging requirement: It specifies what must be captured in phase 1 and is what makes phase 4 check (4) meaningful. Immutability is enforced jointly by the hash chain in phase 1 and the transparency log in phase 3. Usability and scalability are constraints on implementation rather than cryptographic properties, but they are security relevant because a schema that imposes prohibitive overhead will be circumvented. Extensibility is a schema property that ensures that the logging format can accommodate new tool types and new risk indicators without breaking existing verifications. The neutral trust anchor property is what justifies hardware-rooted trust in phase 1 and independent notarization in phase 3: Without it, the schema reduces to a self-signed claim by the designer, which a determined attacker could fabricate.

Achieving tool independence requires that the protocol remain agnostic to which specific software is used at each step of a design workflow. This is accomplished through two design choices. First, the structured provenance envelope uses open, domain-neutral standards that can represent any computational activity regardless of which tool performed it: A Rosetta scoring run and an AlphaFold structure prediction are each recorded as a provenance activity node with typed input and output (I/O) entities, and the schema does not change between tools. Second, recording is implemented as a process that intercepts file I/O and environment variables at the operating-system level rather than requiring integration within each tool, meaning that even legacy tools with no application programming interface (API) or plug-in architecture can be logged without modification to their source code.

Protocol Phases

The following sections describe how the protocol phases are realized using specific open standards and technologies.

Phase 1: Logging

During a design run, the recorder observes each computational event (tool invocation, input file, parameter set, output artifact) and appends a structured record to an append-only local log. Each record includes a cryptographic hash of the previous record, forming a hash chain that makes retroactive insertion or deletion of entries detectable. The recorder operates within a hardware-enforced trust boundary, such as a TEE, such that the private signing key is never accessible to the designer or the host operating system. This property of hardware-rooted trust is what distinguishes the recorder's attestation from a self-signed claim by the designer: It provides strong evidence that the log was produced by a known, unmodified software process rather than by the designer acting unilaterally. Note that software-rooted trust, in which the signing key is protected only by operating-system access controls, may be acceptable in lower-risk deployment contexts but provides weaker guarantees because a sufficiently privileged attacker on the host machine could extract the key.

Phase 2: Sealing

At the conclusion of a design run, the recorder computes a root hash over the full log and produces a signed attestation that binds the root hash to the identity of the recorder process and the hardware environment in

which it ran. This attestation is the primary artifact of the protocol: It is a machine-verifiable claim that a specific log was produced by a specific, unmodified recorder process at a specific time.

Phase 3: Notarization

The designer submits the log and attestation to a notary. The notary independently replays the hash chain to confirm internal consistency, validates the attestation signature, and applies any domain-specific policies—for example, checking that no output sequence exceeds a defined similarity threshold to sequences of known concern. If validation passes, the notary appends an inclusion record to a public, append-only transparency log, obtaining a time stamp and an inclusion proof. The notary returns a signed bundle to the designer that contains the original log, the attestation, and the inclusion proof. The transparency log serves a specific security function: It makes backdating detectable. Because entries in an append-only log are publicly observable and time stamped, a designer cannot later claim that a log was produced at an earlier time than it actually was.

Phase 4: Verification

When submitting a sequence for synthesis or regulatory review, the designer includes the signed bundle alongside the sequence. The verifier checks four things in order: (1) the hash chain is internally consistent, (2) the attestation signature is valid and was issued by a recognized notary, (3) the inclusion proof confirms that the attestation appears in the transparency log at a plausible time, and (4) the sequence in the submission matches the output recorded in the log. A submission that passes all four checks provides the verifier with evidence of nonrepudiation (the designer cannot plausibly deny that the run occurred), tamper evidence (any post hoc modification to the log is detectable), and attribution (the run is linked to an authenticated identity).

Below we revisit each requirement through a security lens, explaining how it directly mitigates threat scenarios involving misuse of AI-bio tools.

- **Completeness:** Every input, parameter, environment detail, and output must be logged.
 - *Security rationale:* Adversaries often exploit *gaps* in provenance. They tweak a single hyperparameter that steers a model toward generating an immunosuppressive peptide or swap a reference database to bias fitness scoring. A forensic investigator who lacks those missing pieces cannot reconstruct intent or demonstrate culpability. Full-fidelity capture, including random seeds, software container hashes, and side-channel artifacts (e.g., GPU driver versions), closes the “excuse window” by eliminating plausible deniability. It also empowers negative verification: proving that a design did not pass through restricted sequences because no such inputs exist in the immutable log (Hayes et al., 2020).
- **Immutability:** Any postrun alteration must be cryptographically detectable.
 - *Security rationale:* In the absence of immutability, a malicious insider can sanitize a log moments before export, eliminating incriminating events while retaining a valid-looking record. Merkle tree–anchored hash chains, countersigned by an external transparency log, force the attacker to compromise *all* participating keys and services simultaneously for a successful attack, a significantly higher bar than editing a local file (Weber et al., 2017).
- **Usability:** Security controls must impose low overhead and require minimal manual effort.
 - *Security rationale:* History shows that scientists will route around burdensome controls. Keeping overhead low ensures comprehensive adoption, turning the audit trail from a niche compliance tool into a ubiquitous layer of defense. High usability also reduces configuration drift that could otherwise create exploitable blind spots (Buckmann et al., 2025).

- **Scalability:** The system must operate on both laptop experiments and million-core cloud clusters.
 - *Security rationale:* A determined actor can shift workloads from regulated corporate servers to personal GPUs or leverage spot instances across multiple clouds to evade detection. A provenance system that functions identically in all execution contexts removes this “jurisdiction shopping” incentive. Moreover, the same tooling must keep pace with automated, AI-driven design-explore loops that generate tens of thousands of candidate biomaterial designs (such as protein structures) per hour; otherwise, attackers could drown the system in data gaps (Felber, 2021).
- **Extensibility:** The schema must evolve without breaking backward compatibility.
 - *Security rationale:* Biosecurity threat intelligence is dynamic. New risk indicators, such as protein language-model capabilities correlated with toxin function, surface regularly. A rigid schema would either freeze in obsolete checks or encourage users to annotate risky fields in ad hoc comments, defeating machine validation (Daum, 2003, Ch. 12).
- **Neutral trust anchor:** Signing keys must reside outside the designer’s control.
 - *Security rationale:* The most damaging scenarios involve privileged users weaponizing legitimate access. Hardware security modules (HSMs), trusted platform modules (TPMs), Intel Software Guard Extensions (SGX) or Amazon Web Services (AWS) Nitro Enclaves, or remote notaries contain features that can prevent these insiders from minting fresh signatures on doctored logs (Aref, Alfughi, and Ouda, 2025; Intel Corporation, 2022; Wang and Wang, 2025). They also enable cross-organizational trust, such that synthesis providers in country A can rely on attestations produced in country B because the root of trust resides in neutral hardware or audited third-party services, not in the designer’s personal key.

Together, these requirements form a mutually reinforcing system: Completeness feeds immutability, usability drives adoption, scalability widens coverage, extensibility future-proofs the schema, and a neutral trust anchor closes insider and supply-chain attack vectors.

Audit Trail Design

The preceding discussion has established *what* the protocol must achieve: six security properties, each grounded in a specific threat scenario, and each mapped to a phase of the protocol workflow. What remains is to specify *how* those properties are realized in practice using concrete, deployable technology. The specific standards and technologies described in this section were selected based on three criteria: maturity (active maintenance and demonstrated deployment at scale in adjacent domains), openness (publicly documented specifications without proprietary lock-in), and composability (designed to interoperate with other components in the stack). The World Wide Web Consortium (W3C) PROV Ontology (PROV-O) was selected as the base provenance vocabulary because it is the only domain-neutral provenance standard with formal W3C endorsement and broad tooling support. RO-Crate was selected as the packaging format because it provides a workflow-run profile specifically designed for computational provenance. In-toto was selected as the attestation framework because it is purpose-built for multistep supply chain verification and has been adopted by the Supply-Chain Levels for Software Artifacts (SLSA) framework, which defines graduated levels of build provenance assurance and serves as the benchmark against which this protocol’s assurance levels should be calibrated. Sigstore Rekord is cited as an example of a mature, publicly operated transparency log; the protocol does not require Sigstore specifically, and any transparency log that is append-only and publicly auditable and provides RFC 3161 time stamps would satisfy the requirement.

The audit trail solution is easiest to understand when visualized in three concentric layers, each answering a different question for reviewers who are concerned with how a particular biomaterial was designed.

Structured Provenance Envelope (Phase 1: Logging)

This layer captures a human-readable diary of the design run in a machine-readable format. Imagine every step one takes in the lab when using an AI-bio tool, such as loading a sequence file, changing a parameter, and pressing the run button, being automatically written into an electronic notebook the moment it happens. The notebook also glues in copies (or cryptographic fingerprints) of every file that was touched. This can label actions, data files, and their relationships. Everything is collaged in a package (e.g., a ZIP file) that keeps the log and its associated files together so that nothing gets lost. With a complete envelope, a regulator can open a single file and see who launched the run, which sequences were fed in, what design settings were chosen (e.g., “maximize thermostability”), and when each stage finished (Lebo, Sahoo, and McGuinness, 2013; Simonyan, Goecks, and Mazumder, 2017).

Cryptographic Sealing (Phase 2: Sealing)

Compute SHA-256 for every log line.

Maintain a hash chain.

Wrap the final root hash in an attestation.

Once the log is written, it needs to be locked so that entries cannot be removed or rewritten. Think of a wax seal on an envelope: If the seal is broken, it is apparent that someone opened the letter. In the digital world, that seal is created with hashes—mathematical fingerprints that change if even a single character changes. Each diary entry includes the fingerprint (SHA-256 hash) of the previous entry, forming a chain. If someone edits line 3, the hash on line 4 no longer matches, and the chain breaks. The *final* hash is then signed with a private key stored in a secure chip (HSM or TPM) or a trusted notary. The signature is wrapped in an attestation, which simply says, “Here’s the diary and here’s my unforgeable promise that it hasn’t changed.” A hostile actor cannot delete the line that says “filtered out 78 sequences because they were too toxic” without the hash chain revealing the cut. The signature proves to a synthesis provider that the log they received is identical to what existed right after the design run finished (Mohanty et al., 2022; Yao et al., 2025).

Transparency Anchors (Phase 3: Notarization)

Sign attestation with an HSM-bound key.

Submit signature to Merkle log; obtain inclusion proof and time stamp.

Even a valid-looking signature might be forged if the attacker stole the key. Transparency logs add an extra, public checkpoint. Picture a town clerk who records every notarized document in a public ledger. Anyone can check the ledger to see whether a given document was indeed recorded on a certain date. The signed attestation is uploaded to a Merkle log (e.g., Sigstore Rekord) (GoogleContainerTools/Distroless Maintainers, undated; Newman, Meyers, and Torres-Arias, 2022). The log immediately issues a receipt, or an inclusion proof, that anyone can later use to confirm that the attestation exists and has not been removed. A malicious insider cannot quietly replace an old, benign signature with a new, doctored one because auditors will look up the original entry in the public log. If it is missing or different, alarms go off.

With these three layers in place, even nontechnical stakeholders (e.g., a biosafety officer, a customs agent) can follow a simple checklist: open the envelope, verify the signature, and look up the receipt in the public log. If all three checks pass, they can accept the design artifact with high confidence that it has not been altered and that the recorded workflow is complete.

Trust Boundary at the Tool Interface

A critical constraint on the protocol concerns the boundary between the AI-bio tool itself and the recorder: The protocol can guarantee the integrity of data only after they have been captured by the recorder, not before. If a tool’s outputs are manipulated before the recorder observes them—for example, by a malicious

process intercepting file I/O on the host operating system—the resulting log will be internally consistent and cryptographically valid but will not accurately reflect what the tool actually computed. This is a fundamental limitation that the protocol does not eliminate; it displaces rather than removes the trust requirement.

Three deployment configurations address this constraint to varying degrees, each with different practical trade-offs. In the strongest configuration, the recorder and the AI-bio tool run together inside the same TEE so that the tool’s outputs are captured before they are ever exposed to the host operating system. This provides the highest assurance but requires that the tool be compiled and attested for the TEE, which is not currently feasible for most existing biodesign software. In the second configuration, the recorder operates as a kernel-level or hypervisor-level monitor that intercepts system calls made by the tool process. This configuration avoids modifying the tool itself but requires elevated privileges and carries its own security assumptions about the integrity of the monitoring layer. In the weakest but most practically accessible configuration, the recorder wraps the tool as a subprocess and observes its declared inputs and outputs via standard file-system and environment-variable interfaces. This last approach is vulnerable to a designer who modifies tool outputs after the run but before the recorder finalizes the log.

Implementers and policy adopters should therefore understand that the assurance level of the audit trail is bounded by the trust configuration in use. A system operating in the subprocess-wrapper configuration provides meaningful evidence of nonrepudiation against casual or opportunistic falsification but may not withstand a determined, technically sophisticated designer with root access to their own machine. Mandating TEE co-execution for high-risk design categories while accepting the subprocess model for routine research is one practical approach to calibrating this trade-off. Any deployment guidance accompanying the protocol should specify which configuration is required for a given risk tier, and synthesis providers should understand what assurance level they are relying on when accepting a submitted bundle.

Trust Architecture and Deployment Configurations

This chapter describes how the protocol phases defined in Chapter 3 are realized across different deployment configurations. Because the recorder must operate in environments ranging from individual researcher workstations to institutional high-performance computing (HPC) clusters to air-gapped secure facilities, no single implementation is appropriate for all contexts. This chapter opens with a description of the parties involved and their specific roles and requirements, then describes three deployment configurations: hardware-bound local signing, remote notary signing, and offline trust-list verification, as variants of the same underlying protocol. It concludes with a discussion of alternative trust architectures, including centralized and federated identity management models, and their trade-offs relative to the fully decentralized baseline. The security of the audit trail is based on two core principles: a hardware-based root of trust for cryptographic signing operations and a public, append-only transparency log for the immutable and verifiable recording of all generated attestations. Across all three configurations, the underlying protocol sequence is the same. During a design run, the recorder observes each computational event, such as tool invocations, input files, parameter sets, and output artifacts, and appends a structured record to a local, append-only log. Each record includes the cryptographic hash of the record immediately preceding it, forming a hash chain in which any retroactive insertion, deletion, or modification is immediately detectable. At the conclusion of the run, the recorder computes a root hash over the full log and produces a signed attestation binding that root hash to the identity of the recorder process and the hardware environment in which it ran. This attestation is then submitted to a notary, which independently replays the hash chain to confirm internal consistency, validates the attestation signature, and applies any domain-specific policies before anchoring the result in a public, append-only transparency log. The resulting signed bundle, which comprises the log, the attestation, and an inclusion proof from the transparency log, is what the designer submits alongside a sequence to a synthesis provider or regulator for verification.

What varies across the three deployment configurations is not this sequence of steps but the trust assumptions underlying each one: specifically, where the signing key resides, who controls it, and what institutional or hardware mechanism prevents the designer from forging or suppressing log entries. In the hardware-bound configuration, that control is provided by a TEE on the designer's machine. In the remote notary configuration, it is delegated to an independent third-party service. In the offline configuration, it relies on a predistributed trust list in place of a live transparency log. Each configuration offers a different point on the trade-off spectrum between assurance strength and practical accessibility, and each is suited to a different deployment context.

Table 4.1 summarizes the four parties identified in Chapter 3, the actors who may fill each role in practice, and the technical and organizational requirements that each role places on its holder. These roles are referenced throughout this chapter as the deployment configurations are described.

In all three primary deployment configurations, the designer's role is identical: to initiate design runs and submit the resulting signed bundle to a verifier.

TABLE 4.1

Roles, Actors, and Requirements for Implementing Verifiable Audit Trails

Role or Actor	Who Fills This Role	Action and Requirements
AI-bio tool end user or designer	Researchers, academics, hobbyists, and private-sector laboratory scientists who use AI-bio tools to generate biological sequences	Download and run VAT software alongside their design tools, generate a cryptographic key pair for signing, register the public key with a certificate authority (this step is automatic in most deployment configurations), initiate design runs, and submit the resulting signed audit bundle to a verifier.
AI-bio tool developer	Frontier labs and commercial developers who create and maintain AI-bio design tools	Provide tool or model access in a form that permits remote or subprocess execution by the VAT recorder, allow the VAT software to observe and log I/Os when the tool runs on developer-controlled servers, and, optionally, expose structured run logs to enable richer provenance capture.
Resource supplier or verifier	Nucleic acid synthesis providers (e.g., International Gene Synthesis Consortium [IGSC] members), regulatory agencies, and journal editors who receive design submissions for review	Receive and verify submitted audit bundles alongside sequence files; confirm hash-chain integrity, attestation validity, and transparency-log inclusion; apply domain-specific screening policies (e.g., sequence-of-concern comparison) based on bundle contents; and make synthesis, export, or publication decisions on the basis of verification outcomes.
Third-party infrastructure operator	Audit software developers (recorder implementers), independent auditors and transparency log operators, certificate authorities, time-stamping authorities	• Recorder developer: build and maintain the VAT wrapper software that captures design-run events and produces signed logs. • Transparency log or notary operator: receive attestations, anchor them in a public append-only Merkle log, issue inclusion proofs, and publish signed root hashes. • Certificate authority: manage the key infrastructure that binds recorder identities to verified credentials, and make public keys available for downstream verification. • Time-stamping authority: issue RFC 3161 time stamps that make backdating of attestations independently detectable.

Verification Workflow

The following verification workflow describes how a verifier—illustrated here using a synthesis provider as the primary example—integrates audit bundle verification into an existing order-review pipeline.

Nucleic acid synthesis providers can integrate audit-trail verification into their existing sequence-screening and order-review pipelines. Synthesis providers represent the most immediate and well-defined use case for audit-trail verification because they already perform sequence screening and have an established operational reason to request provenance documentation. The verification workflow shown in Figure 4.1 uses synthesis providers as the primary illustrative example; the same protocol applies, with appropriate policy adaptation, to other verification contexts, including journal submission, export licensing, and institutional biosafety review.

Table 4.2 maps each step of the synthesis provider verification workflow to the corresponding component of the VAT architecture, showing how the framework integrates into existing order-review pipelines.

Deployment Configuration 1: Hardware-Bound Local Signing

In this configuration, the recorder runs on the designer's local machine or institutional compute node inside a TEE. The recorder role is filled by a local software process; no separate notary organization is required

FIGURE 4.1
Automated Verification Workflow for Synthesis Providers

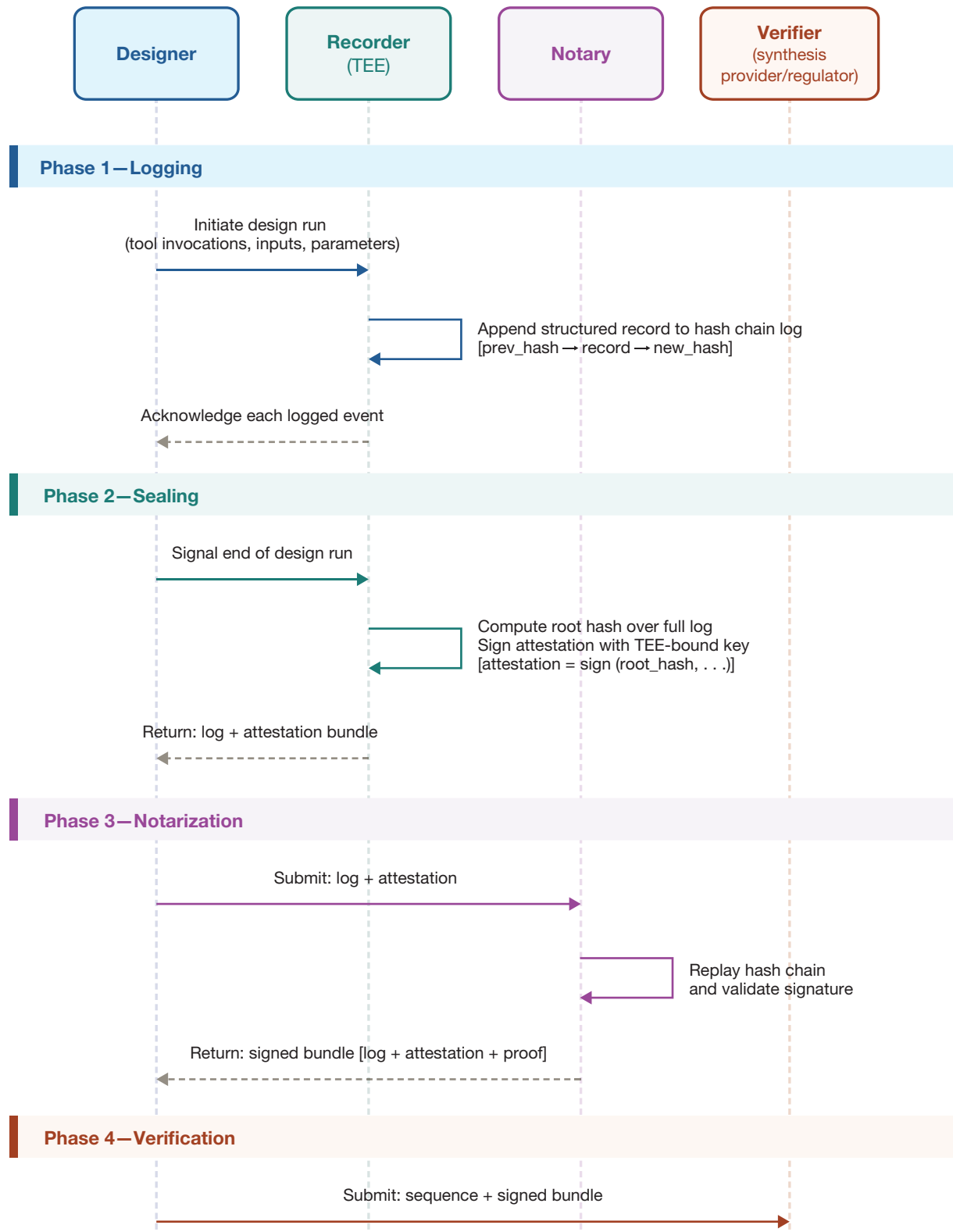


TABLE 4.2

Mapping of Each Verification Workflow Step to the Verifiable Audit-Trail Architecture

Pipeline Stage	Automated Action	Example Data Consumed from Provenance Envelope
1. Intake upload	The customer submits a bundle or package containing, e.g., (a) FASTA or GenBank design files, (b) signed attestation bundle (*.intoto.jsonl.sig), and (c) the ledger of research data and metadata in the bundle or package.	Whole bundle (HTTP/S3 upload)
2. Signature and log check	The verification step confirms the following: • The signature matches an approved key. • Inclusion proof exists in metadata ledger. • The time stamp falls within the policy window (e.g., ≤ 90 days old).	Signature plus inclusion proof
3. Hardware attestation gate	Parse attestation: Accept only hashes on the allow-list or attestation document signed by a public key. Reject if the run was signed by a remote notary not belonging to a hypothetical Biosecurity-Screened Notary Consortium.	platform_attestation, signing_cert_chain
4. Sequence extraction and screening	Use the ledger of metadata and data included in the bundle or package to locate output_type: "nucleic_acid" or "protein_sequence" artifacts; compute SHA-256 and store. Pass sequences through the following: • Search common repositories of controlled sequences for a match. • Perform a Basic Local Alignment Search Tool (BLAST) search against IGSC reference toxin list. • Proprietary sequence-fragment-length-based or embedding-based risk models.	Graph entries for outputs
5. AI-design intent audit (optional)	If the bundle or package contains an extension namespace diffusion or reinforcement_learning, flag for human review. The reviewer inspects parameters and objective fields, e.g., "increase membrane permeabilization," for dual-use research of concern (DURC) keywords.	Extensions, free-text fields
6. Policy decision	If all automated checks pass and no risk keywords are triggered, order progresses to synthesis. Otherwise, escalate to the biosafety officer.	Aggregated verification report

because the TEE hardware itself serves as the trust anchor. This configuration provides the strongest assurance but requires compatible TEE hardware and may not be feasible for some user environments without modification.

The root of trust for all cryptographic operations is established through hardware-bound key generation and storage. Private keys are generated, stored, and used exclusively within the cryptographic boundary of a TEE, such as a physical TPM or a virtualized enclave (e.g., Intel SGX, AWS Nitro Enclaves). This ensures that key material is inaccessible to the host operating system or any colocated processes, mitigating threats from compromised software.

To authorize signing operations, a short-lived certificate is issued to the TEE, contingent on successful remote attestation (a cryptographic signature of its identity and software measurement). This process binds each signature to a hardware identity and a specific, verified software state. Any unauthorized modification (i.e., binary drift) invalidates the certificate. To ensure the post facto integrity and nonrepudiation of the provenance record, all signed attestations are anchored in a public, append-only transparency log.

Transparency Log Integration

A Merkle log is an append-only ledger that stores each new record as a leaf in a Merkle tree (Merkle, 1989). The log publishes (and signs) the evolving root hash; because every parent node is a hash of its children, any deletion or alteration of past entries changes the root and is instantly detectable. Clients can request a compact inclusion proof, a chain of sibling hashes that recomputes the same root offline.

Operational Model

During a design run, the recorder streams each recordable event over transport layer security to a log server that

- assigns a monotonic leaf index
- returns the leaf hash and inclusion proof
- periodically signs the updated root hash with an HSM-protected key and publishes it.

The recorder embeds the returned `leaf_hash_prev` in the *next* line of the local file, chaining the client and server states together.

Table 4.3 provides examples of existing software audit log systems deployed at scale in adjacent domains, demonstrating that the transparency log infrastructure required by this configuration is mature and that the throughput demands of biodesign workflows are well within proven capacities.

The peak laboratory workload for a large pharmaceutical design pipeline, tens of thousands of events per day, sits well within these proven capacities.

Deployment Configuration 2: Remote Notary Signing

In this configuration, the designer’s machine does not require a local TEE. Instead, the recorder produces an unsigned log locally, and a remote notary service acts as the trust anchor. The notary role is filled by an independent third-party service, such as a commercial provider, an institutional operator, or a biosecurity-specific authority, that applies domain policies before signing. This configuration is more accessible than Configuration 1 for researchers who lack TEE-capable hardware, and it enables centralized policy enforcement by a biosecurity authority.

TABLE 4.3
Example of Software Audit Logs Already in Use

Service or Project	Primary Domain	2025 Daily Volume	Notes for Biodesign Context
Sigstore Rekor	Software supply-chain signatures	~5 million entries	Is used by Kubernetes, distroless containers, and Homebrew to store in-toto attestations (Grunert, 2023; Poniszewska-Marańda and Czechowska, 2021)
Google Certificate Transparency (e.g., Argon, Aviator)	Web Public Key Infrastructure (PKI)	>500 million certificates	Demonstrates scalability of public Merkle logs (Merkle, 1989)
Go module SumDB	Language-package checksums	~2 million entries	Shows feasibility for high-frequency developer workflows (Go, 2026)
Debian Reproducible-Builds Log	Linux-distro binaries	Pilot (~10,000 entries)	Is a prototype for binary provenance analogous to protein-design artifacts (Reproducible Builds, undated)
FoldMark	Generalized watermark encoder and decoder with low-rank adaptation (LoRA) modules	Pilot	Protein generative models are fine-tuned with watermark-conditioned LoRA modules that insert a unique identifier code into a protein structure without changing its function (Zhang et al., 2025).
SynthID	Digital watermark using an integrated neural network	Pilot	Neural networks embed and detect watermarks in AI-generated content and can retain metadata related to protein generation process that will be used for verification (Smith, 2024; Wang et al., 2025).

When workstation hardware lacks a TPM or when organizations prefer central oversight, designers can invoke a remote notary, or a policy-aware signing service that owns the private key and decides whether a provenance bundle deserves a signature.

The workflow is as follows:

- The client uploads the unsigned log and root hash package (e.g., in RO-Crate) to the notary's API.
- The notary validates the schema, replays the hash chain, and applies domain policies, e.g., “no peptide has greater than 85 percent similarity to World Health Organization Risk Group 4 toxins.”
- If checks pass, the notary signs the root hash with its HSM-backed key, stores the signature in a Merkle log (e.g., Rekor, Trillian), and returns the JSON web signature (JWS) bundle URL.

Table 4.4 provides examples of existing remote, policy-gated signing services deployed in other high-assurance domains, demonstrating that this configuration has precedent and that the policy-enforcement mechanisms it requires are technically mature.

Deployment Configuration 3: Offline and Air-Gapped Verification

Offline verification (for air-gapped machines operating in physical isolation from any network, including the internet) requires additional setup to enhance security.

An *air gap*, in the context of cybersecurity, refers to the physical separation of a computer or network from any other network, including the internet. This means there are no wired or wireless connections linking it to other systems. The purpose of this isolation is to prevent unauthorized access and protect sensitive data from cyberattacks. This isolation makes it extremely difficult for external threats, such as malware or hackers, to access the system.

TABLE 4.4
Existing Services That Enable Remote Signing

Service	Sector	Under-the-Hood Technology	Relevance to Audit Trails
Sigstore Fulcio	Software	AWS Nitro Enclaves (HSM mode); OpenID Connect identity	Issues short-lived X.509 certificates only after verifying workload identity and policy
AWS Signer plus Nitro Attestation	Cloud continuous integration and continuous deployment	AWS Nitro Enclaves; AWS Key Management Service HSM	Is used by AWS CodePipeline and AWS Proton to sign build artifacts once policy checks pass
Google Cloud Binary Authorization	Kubernetes deployments	Trillian Merkle log; HSM	Blocks container images from running until a remote attester signs them
In-toto and Notary Project (Cloud Native Computing Foundation)	Open source software supply chain	Trillian; HSM	Is the reference implementation of policy-based, remote signing for in-toto statements
Docker or Open Container Initiative Notary v2	Container registries	The Update Framework metadata, SHA and BLAKE digests	Is a converging standard adopted by Harbor and Quay registries
Adobe PKI and PDF Advanced Electronic Signatures (PAdES)	Electronic signatures	X.509 certificates, PAdES, SHA algorithms, Rivest-Shamir-Adleman (RSA) algorithm and Elliptic Curve Digital Signature Algorithm (ECDSA)	Are cryptographic techniques and standards that ensure the authenticity and integrity of digitally signed documents while offline

Although they offer strong protection, air-gapped systems can be cumbersome and limit and compromise functionality of the VAT design because of the lack of network connectivity. It is worth noting that many air-gapped environments do support controlled data transfer via physical media, such as encrypted USB drives or optical discs, which is typically managed through a formal data transfer approval process. In such environments, it would be technically feasible to conduct the online verification workflow with the transparency log submission occurring at a scheduled transfer window rather than in real time. A potential offline approach could operate based off *trust lists*, a trusted list of sequences of concern (SoC) that comes with the structured provenance envelope and is embedded at the root level of the air-gapped machine (Godbold et al., 2022; Godbold and Scholz, 2024; Godbold et al., 2025). The in-toto attestation is validated against this trust list without an internet connection. The offline trust-list approach described here is intended for environments in which even periodic network connectivity is unavailable or in which policy prohibits any outbound data transfer. In other air-gapped contexts, implementers should evaluate whether a delayed-connectivity approach to the standard online workflow would be simpler and provide stronger guarantees than the trust-list approach, which introduces the key management challenges described below.

In the trust-list approach, the recorder produces a signed audit bundle locally, without a live network connection to a notary or transparency log, and verification is performed by the verifier against a predistributed, locally embedded trust list rather than a remote service. The recorder role is filled by a local software process operating under whatever trust boundary is available in the air-gapped environment; because no external transparency log is consulted, the time stamping and backdating detection guarantees provided by Configurations 1 and 2 are replaced by the integrity of the trust list itself and the local system clock. This configuration is the most accessible for environments in which network connectivity is unavailable or prohibited, but it provides weaker security guarantees than the online configurations and introduces key management and list-maintenance challenges that the online configurations do not face.

The verification workflow proceeds as follows. The verifier opens the RO-Crate bundle, extracts the audit log and the in-toto attestation, and recomputes the SHA-256 hash chain over the log records. The recomputed root hash is compared against the value stored in the attestation signature; a mismatch at any point in the chain indicates that log records have been added, removed, or modified since the bundle was sealed. If the hash chain is internally consistent, the verifier checks the attestation signature against a locally stored copy of the signer’s public key certificate, confirming the signer’s identity and the time of signing as recorded in the certificate. Finally, the output sequences extracted from the log are compared against the predistributed SoC trust list embedded in the verification environment. The outcome of these three checks (hash-chain integrity, signature validity, and SoC comparison) is surfaced to the verifier through a simple status display: A green checkmark indicates that the bundle is cryptographically intact, the signer is recognized, and no flagged sequences were detected; a red indicator signals a failure at one or more steps, with the specific failure mode identified so that the verifier can determine the appropriate response.

The trust-list approach carries several limitations that are worth examining in detail because they are not unique to this protocol but are structural features of any list-based biosecurity mechanism applied in an offline context.

The most fundamental limitation is shared with other SoC screening approaches, including the Federal Select Agent Program, disease-specific pathogen surveillance systems, and pathogen-agnostic sequence surveillance frameworks: All of these rely on a defined list of sequences or agents whose properties are known to be dangerous. A list-based approach can flag only what is already on the list. Sequences that are genuinely novel may not match any list entry even if they are functionally equivalent to a listed sequence in terms of hazard potential. This is precisely the scenario that the broader audit-trail framework is intended to address, and it means that the trust-list configuration provides weaker coverage against the highest-risk design category that motivates the protocol. Implementers relying on this configuration should understand that it is

more effective as a check against the use of known restricted sequences in a design workflow than as a general screen for novel harmful designs.

A second limitation is the coordination burden. Several synthesis providers already maintain internal SoC screening lists, and standards bodies, such as IGSC, have developed shared screening frameworks. A trust list deployed in an air-gapped verification environment must remain synchronized with these existing lists to avoid situations in which a sequence passes the offline check but would be flagged by a provider's own screening or situations in which a sequence is on a provider's internal list but absent from the embedded trust list. Maintaining that synchronization across decentralized, air-gapped deployments without a network update mechanism is a nontrivial operational challenge, and the risk that a trust list becomes outdated grows over time as new SoCs are identified. Any deployment of this configuration should specify a formal list-update procedure, including how updated lists are transferred to air-gapped environments, how they are authenticated to prevent substitution of a manipulated list, and how version mismatches between a submitted bundle and the verifier's list are handled.

A third limitation concerns the incentive structure for adoption. Configurations 1 and 2 can, in principle, be enforced by requiring a valid, externally notarized audit bundle as a condition of synthesis order acceptance or regulatory submission: A designer who does not run the recorder simply cannot produce the required credential. The trust-list configuration does not have this property in the same way because the verification step occurs at the air-gapped verifier's end rather than being enforced upstream. There is therefore no mechanism inherent to this configuration that creates a positive incentive for the designer to install and run the audit software in the first place. Adoption in this configuration is more likely to depend on institutional mandate, contractual obligation, or professional norm than on a technical gatekeeping mechanism. This is not a reason to reject the configuration for contexts in which it is the only feasible option, but it does mean that policy adoption strategies appropriate for Configurations 1 and 2 would need to be supplemented by out-of-band enforcement mechanisms if applied to air-gapped settings.

Alternative Trust Architectures and Design Trade-Offs

The architecture described in this paper is not the only design capable of satisfying the stated security properties, and it is worth considering simpler alternatives explicitly to justify the chosen approach and acknowledge the contexts in which a lighter-weight design might be preferable.

The most significant simplification would be the adoption of a centralized trust authority, an institution that manages all signing keys, maintains the transparency log, and serves as the sole notary for the ecosystem. This is analogous to the role played by certificate authorities. Under a centralized model, the designer authenticates to the central authority at the start of each design run, the authority issues a short-lived session credential, and all log attestations are signed under that credential and verifiable against a single published root. This model eliminates the complexity of distributed key management, makes revocation straightforward, and simplifies the verifier's trust decisions: A verifier needs to trust only one root rather than a federation of notaries.

The trade-off is that a centralized authority introduces questions about the operation and accountability of this authority. If the authority is compromised, coerced, or simply unavailable, the entire verification ecosystem fails. For a biosecurity application with global reach and potential adversarial pressure, this concentration of trust is a significant vulnerability.

A middle path, and one closer to existing practice in high-assurance environments, is federated identity management, in which institutional identity providers (e.g., universities, research institutions, commercial labs) issue credentials to their own researchers, and those credentials are mutually recognized by notaries and verifiers through a federated trust agreement. This preserves decentralization at the key-management

level while reducing the complexity faced by individual designers, who authenticate once to their home institution rather than managing their own cryptographic keys. This approach is worth developing further in future work, particularly because it maps naturally onto existing research computing infrastructure and would lower the barrier to adoption for academic users.

The architecture proposed in this paper is best understood as a maximally decentralized baseline: It makes no assumption that any central authority exists and is therefore compatible with early-stage deployment before governance structures are established. As the ecosystem matures, elements of the design (particularly key management and notary accreditation) could be progressively centralized or federated without changing the underlying protocol properties.

Cross-Tool Generalizability and Common Workflows

The VAT protocol is designed to be tool independent: It does not require modification to any AI-bio tool's source code, integration with a tool's API, or cooperation from the tool's developer. Instead, the recorder operates at the level of the computing environment in which a tool runs, observing I/Os through standard operating system interfaces that are available regardless of which tool is in use. This chapter specifies what the protocol does and does not require of a tool environment, what the recorder observes at each stage of a design run, and what obligations tool owners and institutional administrators should understand if they wish to support or mandate audit-trail adoption. Details of the specific tools in use as of this writing in 2026 and the typical workflows they support are provided in the appendix for readers who want to understand how these requirements map onto existing practice.

What the Protocol Requires of a Tool Environment

The recorder needs three things from the computing environment in order to produce a complete and trustworthy audit log.

- **Observable file I/O.** The recorder must be able to observe which files a tool reads at the start of a run (e.g., input sequences, parameter files, reference databases) and which files it writes at the end (e.g., output sequences, structure files, score files). On standard operating systems, file reads and writes are mediated by the kernel and can be monitored by a process with appropriate permissions without any cooperation from the tool itself. This is the same mechanism used by backup software, antivirus scanners, and software development profilers. The recorder records the name, size, and cryptographic hash of each observed file at the moment it is read or written, creating a tamper-evident record of what data entered and left the tool.
- **Accessible environment variables and process metadata.** The recorder must be able to read the environment in which the tool process runs: the user identity, the tool's executable path and version string, the command-line arguments passed to the tool, and the hardware and operating system identifiers of the machine. This information is available through standard process inspection interfaces on all major operating systems and does not require elevated privileges beyond those already held by the user running the tool. It provides the identity and context fields that allow a verifier to answer the following question: Who ran this tool, on what machine, with what version, and with what configuration?
- **A hardware or service trust boundary for the signing key.** The recorder must have access to a signing key that is outside the designer's direct control, so that the designer cannot sign a fraudulent log. This may be provided by either a TEE on the local machine or a remote notary service. The tool itself plays no role in this; the requirement falls entirely on the deployment infrastructure.

Successful implementation requires addressing both technical hurdles and user adoption challenges. The following priorities are proposed not as the only viable path to deployment but as the approaches most likely

to accelerate adoption given the specific barriers identified in this paper. Each is motivated by evidence from analogous technology deployments; each also has alternatives that may prove preferable depending on how the governance landscape develops.

- **Reducing user friction through tool integration.** The primary adoption risk for any audit requirement is that researchers route around it, either by using unaudited tools or by generating logs post hoc in ways that undermine their integrity. Evidence from analogous mandates, such as electronic lab notebook requirements and institutional review board protocol registration, consistently shows that compliance rates are highest when the required behavior is a near-automatic by-product of the normal workflow rather than an additional step (Hardwicke et al., 2018; Kidwell et al., 2016). This motivates prioritizing seamless integration into existing tools over stand-alone audit clients and suggests that adoption incentives (such as expedited synthesis review for audited submissions) are likely to be more effective than punitive requirements in the near term.
- **Open-source reference implementation versus standardization through a consortium.** Standardization efforts typically require multiple reference implementations before a standard can be finalized and tested for interoperability. Releasing an open-source implementation first allows the community to identify practical gaps in the protocol specification before those gaps are frozen into a standard. IGSC and analogous bodies would be the appropriate vehicles for eventual standardization, and early engagement with those bodies, to ensure that the reference implementation is designed with their operational constraints in mind, is a prerequisite for the standard ultimately being adopted by synthesis providers. If key stakeholders prefer to begin with a consortium-led standardization process rather than a reference implementation, that approach is viable but carries the risk of producing a standard that is technically sound but operationally burdensome and therefore resisted by the researchers it is meant to cover.
- **Security testing methodology.** Rigorous formal or semiformal analysis of the cryptographic protocol itself, verifying that the composition of hash chaining, attestation, and transparency logging actually achieves the claimed security properties under the stated threat model, should precede any implementation-level testing.
- **Long-term data management.** The challenges of storing and querying large cryptographically linked audit trails at scale are not unique to this application; they are well studied in the context of blockchain systems, certificate transparency logs, and large-scale provenance systems in scientific computing. Rather than treating long-term data management as a research problem to be solved from scratch, future development should survey existing solutions in those adjacent fields and evaluate their applicability before investing in bespoke infrastructure.

What the Protocol Does Not Require

The protocol explicitly does not require any of the following, and tool developers and institutional administrators should understand these as nonrequirements when evaluating adoption:

- The protocol does not require access to a tool’s source code. The recorder observes a tool’s behavior through external interfaces, not by instrumenting the tool’s internals. A closed-source commercial tool can be audited to the same degree as an open-source one.
- The protocol does not require a tool to expose an API or logging interface. Some tools already produce machine-readable run logs; where they do, the recorder can optionally ingest these for richer provenance. But the protocol’s minimum requirements are satisfied by file-system observation alone, without any tool-side cooperation.

- The protocol does not require tool modification or recompilation. The recorder is a separate process that wraps or monitors the tool process. Deploying the recorder does not change the tool’s behavior, its output files, or its performance characteristics in any way that the researcher would observe.
- The protocol, in the basic deployment configuration, does not require network connectivity during the design run itself. Log entries are written locally; submission to a notary and transparency log can occur after the run is complete. The exception is a streaming transparency log configuration, which does require connectivity, but this is an optional higher-assurance deployment variant rather than a baseline requirement.

What Tool Owners Should Do

Although the protocol does not require tool developer cooperation, tool owners can meaningfully improve audit-trail quality and adoption in several ways.

Tool owners who expose structured, machine-readable run logs (for example, in JSON or another parsable format) allow the recorder to capture richer provenance than file-system observation alone provides. Specifically, intermediate computational states, such as model weights used, scoring function parameters, or iteration counts, are often not reflected in input or output files but are captured in tool-internal logs if those logs are accessible. Tool owners who wish to support audit-trail adoption should consider publishing a log schema documenting which fields are available and in what format so that recorder implementations can be written to ingest them.

Institutional administrators who deploy tools on shared compute infrastructure (e.g., HPC clusters, cloud environments, container orchestration platforms) are in a strong position to mandate recorder deployment as a condition of tool access without requiring individual researchers to configure anything. Container-based deployment, in which the recorder is included in the container image alongside the tool, is likely the path of least friction for institutional adoption because it ensures consistent configuration across all users and removes the installation burden from individual researchers.

Tool developers who are willing to go further can integrate native attestation support—for example, by having the tool itself sign a manifest of its inputs, parameters, and outputs using a TEE-bound key at the end of each run. This provides stronger provenance than external observation alone because it captures information that is internal to the tool and not visible through file-system interfaces. However, this level of integration is not required by the protocol and should be understood as a best-practice enhancement rather than a baseline expectation.

Cross-Tool Workflows

A single biological design project frequently involves multiple tools used in sequence: a structure prediction tool to generate initial candidates, a sequence optimization tool to refine them, and a validation tool to filter the results. The protocol handles multitool workflows naturally, provided that the recorder remains active across all tool invocations within a session. Each tool invocation is recorded as a separate activity node in the provenance graph, with edges connecting the output files of one tool to the input files of the next. The resulting audit bundle describes the complete multitool workflow as a single cryptographically sealed record rather than a collection of separate per-tool logs that a reviewer would need to manually correlate.

The practical requirement that this imposes on deployment is that the recorder must be activated at the session level—for example, when a researcher logs into a compute environment—rather than being invoked separately for each tool. Institutional deployment via container or HPC job wrapper is well suited to this;

individual workstation deployment requires that researchers understand the recorder as a session-level process rather than a per-tool one.

For the benefits of an audit log to be fully realized, the software that generates the log should work across computing environments and with any tool used in the design of biological material. To that end, we analyzed several tools to identify commonalities that should be incorporated into any audit-trail software.

The protocol is tool independent in the sense that it does not require modification to any specific AI-bio tool; the recorder operates at the level of the computing environment rather than within any tool. It is also capable of capturing multitool workflows, such as a single design session that moves from Rosetta to ProteinMPNN to AlphaFold, within a single audit bundle, provided that the recorder remains active across all tool invocations.

Achieving cross-tool generalizability requires that the audit-trail architecture remain agnostic to the specific software used at each step of a design workflow. This is accomplished through two design choices. First, the structured provenance envelope uses open, domain-neutral standards that can represent any computational activity regardless of which tool performed it. A Rosetta scoring run and an AlphaFold structure prediction, for example, are each recorded as an activity node with typed I/O entities; the schema does not need to change between tools, only the values within it. Second, the audit recorder is implemented as a thin wrapper or sidecar process that intercepts file I/O and environment variables at the operating system level rather than requiring native integration within each tool. This means that even legacy tools with no API or plug-in architecture, such as stand-alone Rosetta executables or locally hosted ESMFold instances, can be logged without any modification to their source code. Together, these choices allow the same audit envelope, cryptographic sealing, and transparency-log submission process to apply uniformly across the full variety of tools described in the appendix and to any new tools that enter the field in the future.

Policy Considerations and Conclusion

The preceding chapters have established the verification gap and proposed a technical solution to it via secure audit trails. This chapter details some implications of a verifiable audit chain and the challenges to adoption within the research enterprise.

To bridge the verification gap in protein design, we recommend advancing the conceptual framework presented here toward a scalable, fully developed audit-trail platform. We do not present a working implementation; rather, we propose a protocol design.

Policy Implications of Verifiable Audit Trails

The technical protocol described in this paper has implications that extend beyond implementation choices and into the structure of biosecurity governance. This chapter articulates those implications, as follows, before turning to the question of how they might be acted on:

- *Verification shifts from sequence-centric to process-centric screening.* Existing biosecurity screening by synthesis providers focuses primarily on the output: The sequence submitted for synthesis is compared against databases of SoCs. This approach is inherently reactive and is increasingly strained by the combinatorial space of novel sequences that generative AI tools can produce. A VAT does not replace sequence screening but changes its evidentiary basis. A verifier can now ask not only “Does this sequence match a known dangerous sequence?” but “Was this sequence generated by a process that included optimization objectives consistent with legitimate research?” This is a meaningful expansion of the information available at the point of screening, and policy frameworks governing synthesis providers should be updated to reflect it, by both specifying what audit information providers may request and establishing what inferences they are permitted to draw from it.
- *Nonrepudiation creates new legal and regulatory surface.* An audit trail that is cryptographically bound to an authenticated identity changes the legal character of the design process. Where a designer can plausibly deny that a particular sequence was intentionally optimized for harmful properties, a verified audit log makes such denial much harder to sustain. This has implications for how liability is allocated in cases of dual-use research, how export control compliance is demonstrated for digital biological artifacts, and how research misconduct investigations are conducted. Policymakers should consider these implications proactively rather than waiting for the legal questions to be resolved through litigation after the fact.
- *An audit requirement has distributional consequences that can be addressed in policy design.* A mandate requiring VATs as a condition of synthesis access would not affect all researchers equally. Well-resourced commercial and academic institutions with dedicated information technology infrastructure are better positioned to absorb the overhead of audit tooling than individual academic researchers, early-career scientists, or researchers in lower-income countries. If an audit requirement disproportionately burdens the latter groups without meaningfully constraining the actors who pose the highest

risk, who are likely to be well resourced, it risks reducing the diversity of the research ecosystem without achieving its biosecurity objective. Policy design should therefore include provisions for subsidized or institutionally managed audit infrastructure and should evaluate compliance burden explicitly as part of any regulatory impact assessment.

- *Auditability is not equivalent to oversight.* A VAT establishes that a record exists and has not been tampered with. It does not, by itself, ensure that anyone with appropriate expertise reviews that record, that review happens in a timely way, or that reviewers have the analytical tools to identify risk indicators within a log. The policy infrastructure required to make audit trails effective, including trained reviewers, analytical software, defined risk thresholds, and clear escalation pathways, would be developed alongside the technical infrastructure in an implementation plan. Deploying audit logging without the corresponding review capacity would create the appearance of oversight without the substance.
- *The governance question is prior to the technical question.* The protocol proposed here is technically feasible, but its deployment at meaningful scale requires answers to governance questions that are not technical in nature: Which institution or institutions should operate the transparency log? Under what legal authority can a synthesis provider decline an order on the basis of audit log content? How are disputes about log interpretation adjudicated? How is the system governed to prevent it from becoming a tool of surveillance of legitimate research rather than a check on misuse? These questions will determine whether the technical system achieves its intended purpose, and they require engagement from policymakers, legal scholars, research institutions, and civil society representatives alongside the technical community. The recommendations that follow are therefore best understood not as an implementation roadmap but as inputs to a broader governance process that has yet to be convened.

Recommendations

Our conclusions support the following recommendations, each of which is directed at a specific actor and grounded in the analysis:

- *Biosecurity policymakers and funders should commission a governance design process before investing further in technical development.* The analysis identifies the following as binding constraints on deployment: governance gaps concerning who operates shared infrastructure, how disputes are adjudicated, and how the system is prevented from becoming a surveillance tool. Technical development that proceeds without resolving these questions risks producing a system that is cryptographically sound but institutionally undeployable. A structured process involving synthesis providers, research institutions, legal experts, and civil society representatives should precede significant investment in production infrastructure.
- *Synthesis provider consortia, including IGSC, should evaluate whether audit-trail submission could be incorporated into existing screening workflows as a voluntary pilot.* The analysis shows that the protocol is compatible with existing order-review pipelines and that the verification steps it requires are analogous to screening that is already performed. A voluntary pilot with willing providers and a defined cohort of research institutions would generate empirical evidence about operational burden, false-positive rates in policy-gated notarization, and researcher compliance behavior—evidence that is absent in the existing infrastructure and that would be required to support any mandatory requirement.
- *Regulatory agencies with jurisdiction over dual-use research, including the National Institutes of Health (NIH) and potentially export control authorities, should assess whether the protocol's nonrepudiation properties create a sufficient evidentiary basis to extend existing DURC frameworks to in silico protein*

design. The analysis shows that a verified audit log materially changes the information available to regulators at the point of screening. However, the legal and regulatory implications of this change, including questions of liability, evidentiary standards, and jurisdictional reach over digital biological artifacts, have not been analyzed for this paper and require dedicated legal and policy analysis before any regulatory action is taken.

- *Software developers and AI-bio tool creators should engage early with the protocol specification to identify integration constraints.* The analysis identifies tool-level integration, specifically the secure capture of tool outputs by the recorder, as the point of highest practical uncertainty in the protocol. Developers of widely used platforms, such as AlphaFold, RoseTTAFold, and ProteinMPNN, are best positioned to assess whether TEE co-execution or kernel-level monitoring is feasible within their architectures and to identify the integration path of least resistance. This engagement should occur before a reference implementation is finalized, not after.
- *Any deployment of the protocol should be accompanied by an explicit specification of the assurance level in use and a corresponding statement of the residual risks.* Our analysis shows that the protocol's security properties vary significantly across deployment configurations and that the weakest configuration, though practically accessible, provides guarantees that may not be adequate for high-risk design categories. Policymakers, synthesis providers, and researchers relying on audit bundles should understand what configuration produced a given bundle and what the limits of the resulting assurance are. A tiered assurance schema should be developed, and standardizing the communication of assurance levels should be a priority in any pilot deployment.

Organizational Roles and Actionable Steps

To realize a robust biosecurity verification framework, specific organizations could coordinate their efforts across the software, regulatory, and commercial sectors, such as the following roles and steps:

1. Software developers and AI-bio tool creators
 - a. Commercialization: Further develop the audit-trail concept into a fully fledged platform integrated with leading biodesign tools.
 - b. Harmonization: Adopt a uniform information model to ensure that provenance data are not only syntactically correct but semantically unambiguous across different platforms.
 - c. Reference implementation: Provide open-source software development kits to democratize responsible innovation.
2. Federal funding and scientific agencies (e.g., NIH, National Institute of Standards and Technology [NIST])
 - a. Financial support: Subsidize the technically complex and costly development of audit platforms, as the positive implications for biosecurity justify government investment.
 - b. Regulatory guidance: NIH or NIST could support adoption by recommending or regulating use.
 - c. Grant compliance: Expand the definition of DURC to cover in silico protein engineering, requiring signed provenance bundles as a condition of grant compliance.
3. Gene synthesis providers and consortia (e.g., IGSC)
 - a. Standard setting: IGSC could set standards regarding audit trails for novel biomolecules.
 - b. Incentivization: Providers could expedite synthesis orders that provide an audit or move toward requiring audits for all orders.
 - c. Integration: Integrate audit-trail verification directly into existing sequence-screening and order-review pipelines.

4. Export control and customs authorities
 - a. Verification: Use signed, hardware-attested logs to obtain machine-verifiable evidence that exported digital artifacts match their stated benign classifications.
 - b. Policy enforcement: Consider “no-log, no-export” policies whereby cloud providers prevent the download of designs until an approved notary has signed the log.

Challenges

Although the protocol described provides a robust technical foundation for verifiable provenance, its real-world deployment faces nontrivial challenges. Continuous cryptographic operations proposed here, such as hashing event data, maintaining Merkle chains, and performing remote attestations, impose a nonzero performance and financial tax on design runs. For high-throughput screening, this overhead could become an impediment. Moreover, the sustainment of a log archive represents a substantial data storage cost for providers and possibly for some researchers as well. Future research should address the long-term challenges of storing, querying, and managing these massive, cryptographically linked audit trails.

The flip side of that technical challenge is the human element. Users may hesitate to use any audit software. Therefore, it is essential to ensure as little friction as possible for the user to facilitate easy adoption by the scientific community. If the system is difficult to install, configure, or use, researchers will bypass it. Successful deployment hinges on seamless integration into the tools used by scientists to reduce friction (e.g., allowing automatic configurations) and make generating a VAT an automatic by-product of the research. It would also be beneficial to communicate with researchers across the ecosystem to understand their concerns. Different institutions may have different concerns about how this system will affect their workflow. Communication would allow opportunities to either assuage those concerns or modify the audit software to mitigate them.

The scalability note in Chapter 4, which mentions the throughput of a large pharmaceutical design pipeline, warrants closer examination in the context of AI-accelerated biology. The cited estimate of tens of thousands of design events per day reflects current use patterns for traditional computational pipelines, in which each design run involves significant human setup time and expert interpretation. AI-driven protein-design tools change this assumption materially. A single researcher using a diffusion-based generative model can produce hundreds of thousands of candidate sequences in a single run; at an institutional or commercial scale, daily event volumes could plausibly reach the hundreds of millions. An order-of-magnitude increase in design throughput, well within the trajectory suggested by existing trends in AI capability and accessibility, would require a corresponding increase in logging, hashing, and transparency-log ingestion capacity that this paper does not address.

Several aspects of the protocol are more sensitive to this volume increase than others. Hash-chain computation scales linearly with event count and is unlikely to be a bottleneck. Transparency log ingestion, however, involves network round trips and server-side Merkle tree updates; at very high throughput, batching strategies would be required, which introduces latency between event occurrence and log inclusion that may weaken the time-stamping guarantee. Notary-side policy evaluation, particularly sequence similarity screening, is computationally expensive and does not scale linearly. At high volume, it would require either approximate methods or tiered screening, both of which carry accuracy trade-offs that have biosecurity implications.

There is also a storage dimension that this paper does not fully address. A complete, cryptographically linked log for a large generative design run could be gigabytes in size. Long-term retention of such logs, which may be required for regulatory or investigative purposes, raises questions about who bears the storage cost, how long logs must be retained, and how archived logs are made queryable without compromising their integrity. These are not purely technical questions; they have significant implications for who can practi-

cally afford to comply with an audit requirement and, therefore, for whether such a requirement would have its intended effect across the full variety of actors it is meant to cover, including well-resourced commercial actors and individual academic researchers alike.

A final apparent challenge that remains is cross-tool harmonization, both with extant biodesign software and flexibility for future developments. Without a uniform information model adopted by tool developers, provenance data may consist of syntactically correct yet semantically ambiguous fields, diminishing the effectiveness of machine-readable audit mechanisms.

A layered audit-trail system is technically feasible with the existing open standards and cloud infrastructure. The primary hurdles are organizational, including standard adoption and hardware-attestation rollout for users of AI-bio tools. However, the resulting ability to provide security assurances to bioeconomy stakeholders may justify early investment.

Commonly Used Biodesign Tools and Typical Workflows

This appendix describes the AI-bio tools most commonly used in protein-design workflows and illustrates how these tools are combined in typical research settings. This material is provided to help readers understand the practical context in which the VAT protocol would be deployed. For the protocol's requirements on these tool environments, see Chapter 4.

Traditional and AI-Driven Design Tools

Traditional protein-design tools focus on using physics-based modeling and search algorithms to identify optimal amino acid sequences that would fold into desired structures. As the name suggests, *AI-driven protein-design tools* use AI or other machine learning models. In some instances, this entails inverting a model, whereby software designed to predict protein structures from sequences then solves the inverse problem (e.g., generating sequences for target structures). Both categories present the same underlying verification challenges, but AI-driven tools have experienced substantial growth in capability and accessibility in the past ten years and are the primary driver of the verification gap described in Chapter 2.

Rosetta is one of the most established and widely used computational platforms for protein design. It can perform de novo protein design, enzyme design, ligand docking modeling, and structure prediction of biological macromolecules and macromolecular complexes. Rosetta's approach incorporates physics-based scoring functions that model van der Waals interactions, electrostatics, solvation, and entropy. It uses algorithms for optimizing side-chain conformations and backbone structures. Rosetta is one of the most versatile and long-standing protein-design products, so it serves as an effective baseline for traditional design workflows (Watson et al., 2023).

Protean 3D provides functionality that is often used for site-directed mutagenesis, in which researchers attempt to predict which combinations of mutations will produce the most-stable proteins. This enables workflows for inserting user-defined amino acid substitutions into known structures, scanning across selected regions to identify stability hot spots, and calculating rotamer conformations to understand where the lowest predicted energy changes are taking place. Protean 3D is often used in conjunction with DNA engineering tools, such as SeqBuilder Pro and other tools designed by DNASTAR, creating an integrated workflow and ecosystem that ranges from protein design to experimental implementation (Burland, 2000).

AlphaFold is the archetypical AI-driven design software following several high-profile publications and its creators being awarded the 2024 Nobel Prize for Chemistry. AlphaFold is used for generating initial novel structures to start design efforts and for filtering down candidates. For instance, one approach using a selection threshold of an average interchain predicted aligned error less than 10 yielded eight- to 30-fold higher success rates for later binder designs (Bennett et al., 2023). AlphaFold has enabled integrative approaches combining computational predictions with experimental data, such as fitting AlphaFold models into cryo-electron microscopy density maps to elucidate complex structures. One strong example of this practice is

work done to reconstitute the majority of the nuclear pore complex (Fontana et al., 2022; Jumper et al., 2021a; Jumper et al., 2021b).

ESMFold is another AI-based structure prediction tool that has been adapted for protein design through a process called *inversion*. Using an inverted ESMFold model called ESM-Design, sequences with profiles that are both native-like and likely to be functionally expressed in cells can be generated (Jeliazkov, del Alamo, and Karpiak, 2023; Manfredi et al., 2024; Zalewski, Wallner, and Kmiecik, 2025). ESM-Design (James et al., 2023; Manfredi et al., 2024) incorporates a protein language model that helps produce more native-like sequences compared with AlphaFold-based models. The method can hallucinate proteins with very high per-residue confidence scores (predicted local distance difference test values greater than 90) within 100 to 300 optimization steps. Sequence initialization plays a large role in convergence, with discrete amino acid sampling producing the most natural-looking sequences. It should be noted that ESM-Design has significant computational limitations and requires extensive memory resources that prevent proteins larger than 150 residues from being modeled (James et al., 2023; Manfredi et al., 2024).

RoseTTAFold has emerged as another deep learning tool and uses a three-track neural network that simultaneously processes sequence patterns, amino acid interactions, and possible three-dimensional conformations. Similarly to AlphaFold, RoseTTAFold has been used to filter protein designs in computational pipelines and can compute structures for proteins with limited experimental data. Its ability to rapidly build models of complex biological assemblies makes it valuable in structure-guided design approaches (Watson et al., 2023).

Typical Workflows

Stability Enhancement Workflow

A common protein-design objective involves enhancing stability through targeted mutations, in the following workflow:

- Hot spot identification: A designer may conduct computational alanine scanning to identify residues that contribute significantly to folding energy and stability.
- Mutation prediction: Rosetta would then be used to evaluate potential substitutions that may increase stability.
- Experimental validation: Selected mutations are tested experimentally, with results potentially feeding back into computational models.

De Novo Protein-Design Workflows

Creating an entirely new protein structure may follow this workflow:

- Topology selection: Designers choose target fold architecture, drawing from prior literature or research.
- Backbone generation: Rosetta generates idealized backbone conformations.
- Sequence design: Specialized algorithm programs, such as ProteinMPNN, may generate sequences predicted to fold into the target structure.
- Structure validation: AI prediction tools, such as AlphaFold, would then verify that a design is likely to fold as intended.
- Experimental characterization: Biophysical methods confirm the structure and function.

Protein-Protein Interface Design

Design of protein interactions follows a specialized workflow:

- Target selection: Rosetta identifies binding partners and interaction surfaces.
- Interface modeling: Designers stay in Rosetta to use its computational prediction power to examine potential interaction geometries.
- Hot spot identification: Designers determine key residues that drive binding, again in Rosetta.
- Sequence optimization: Specialized sequence optimization programs, such as ProteinMPNN, redesign interface residues to enhance affinity and specificity.
- Validation: AI prediction tools, such as AlphaFold, are used to predict alignment error to evaluate designs.

Abbreviations

AI	artificial intelligence
AI-bio	artificial intelligence-enabled biological design
API	application programming interface
AWS	Amazon Web Services
BCO	BioCompute Object
CUDA	Compute Unified Device Architecture
DURC	dual-use research of concern
ECDSA	Elliptic Curve Digital Signature Algorithm
FASTA	FAST-All
GeoIP	Geographical Internet Protocol
GPU	graphics processing unit
HPC	high-performance computing
HSM	hardware security module
IGSC	International Gene Synthesis Consortium
I/O	input and output
IP	internet protocol
JWS	JSON web signature
LoRA	low-rank adaptation
MSA	multiple sequence alignment
NCBI	National Center for Biotechnology Information
NIH	National Institutes of Health
NIST	National Institute of Standards and Technology
OIDC	Open Identification Connect
ORCID	Open Researcher and Contributor ID
PADES	PDF Advanced Electronic Signatures
PDB	Protein Data Bank
PKI	Public Key Infrastructure
PROV-O	PROV Ontology
RSA	Rivest-Shamir-Adleman (algorithm)
SBOM	software bill of materials
SGX	Software Guard Extensions
SHA	secure hash algorithm
SHA-256	secure hash algorithm 256-bit
SLSA	Supply-Chain Levels for Software Artifacts
SoC	sequences of concern
TEE	trusted execution environment
TPM	trusted platform module
VAT	verifiable audit trail
W3C	World Wide Web Consortium

Glossary

amino acid sequence: The ordered string of chemical building blocks called *amino acids* that make up a protein. Just as letters in a specific order form words with specific meanings, amino acids in a specific order cause a protein to fold into a specific three-dimensional shape and perform a specific biological function. In protein design, generating or modifying an amino acid sequence is the primary computational task (Jumper et al., 2021a).

attestation: A cryptographically verifiable claim that a specific, unmodified piece of software is running in a specific hardware environment. Attestation allows a remote party, such as a notary or verifier, to confirm that the recorder process has not been tampered with, without needing physical access to the machine on which it runs (Ménétrety et al., 2024). See also *remote attestation* and *trusted execution environment*.

BioCompute Object (BCO): A standardized format for describing and communicating the steps of a computational biology workflow, developed to support reproducibility and regulatory review. In the context of this protocol, the BCO's provenance and execution domain fields provide a schema for recording design run metadata in a structured, machine-readable way (Alterovitz et al., 2018).

de novo protein design: The computational design of a protein with a structure or function that does not exist in nature, as opposed to modifying an existing natural protein. De novo design is one of the higher-risk applications of AI-bio tools from a biosecurity perspective because it can produce sequences that do not resemble any known natural sequence and therefore may evade screening methods based on sequence similarity to known pathogens (Watson et al., 2023).

dual-use research of concern (DURC): Research that, while conducted for legitimate scientific purposes, could be misused to pose a threat to public health, safety, or national security. NIH maintains a formal DURC policy that focuses on laboratory manipulation of specific high-consequence pathogens; we recommend consideration of whether in silico protein design using AI tools should be brought within the scope of that framework (Imperiale and Casadevall, 2015).

FAST-All (FASTA): Standard DNA file type containing the nucleic acid bases (Pearson and Lipman, 1988).

GeoIP: An address indicating the geographical location of the IP address used (Wang and Simmons, 2006).

hardware security module (HSM): A dedicated physical device or a protected region within a processor designed to generate, store, and use cryptographic keys without exposing the key material to the host system. HSMs are used in applications in which the integrity of a signing key must be guaranteed even if the surrounding software environment is compromised. In this protocol, notary services use HSM-backed keys to sign attestations (Aref, Alfughi, and Ouda, 2025).

hash chain: A sequence of records in which each record contains the hash of the record immediately preceding it. Because each record's hash depends on all previous records, inserting, deleting, or modifying any record breaks the chain at that point and makes the alteration immediately detectable. Hash chains are the primary mechanism for tamper evidence in the logging phase of this protocol (Lefevre and Mennink, 2024).

hash function or hash: A mathematical operation that converts any input, such as a file, record, or sequence, into a short, fixed-length string of bits called a *hash* or *digest*. Hash functions are designed such that the same input always produces the same output, but any change to the input, however small, produces a completely different output. This makes hashes useful as tamper-evident fingerprints. SHA-256 is a widely used hash function that produces a 256-bit digest; it is the specific function used in the hash-chaining component of this protocol (Appel, 2015).

inclusion proof: A compact cryptographic proof, derived from a Merkle tree, that a specific record is present in a transparency log. An inclusion proof allows a verifier to confirm that a particular attestation was submitted to the log at a particular time, without needing to download or inspect the entire log (Laurie, 2014).

in-toto: A framework for supply chain security that allows each step in a multistep process to produce a signed attestation describing what it did, what inputs it received, and what outputs it produced. The resulting chain of attestations allows a verifier to confirm that a final artifact was produced by the expected sequence of steps. In this protocol, in-toto attestations are used to wrap the root hash of the audit log (Torres-Arias et al., 2019).

Merkle log: An append-only ledger whose entries are leaves of a Merkle tree, enabling compact inclusion proofs and making any modification or deletion cryptographically detectable (Laurie, 2014).

protein folding: The process by which a chain of amino acids spontaneously adopts a specific three-dimensional structure. A protein's structure determines its function; predicting how a given sequence will fold is therefore a central challenge in both biological research and biosecurity risk assessment. AI tools, such as AlphaFold, have dramatically improved the accuracy and speed of structure prediction (Jumper et al., 2021a).

remote attestation: Attestation performed across a network connection, allowing a remote party to verify the identity and integrity of a software process running on a machine that it does not physically control. Remote attestation is the mechanism by which a notary can confirm that a submitted log was produced by a genuine, unmodified recorder process running inside a TEE rather than by a designer-controlled process mimicking the recorder (Ménétrety et al., 2024).

RO-Crate: A community standard for packaging research data, metadata, and computational workflows into a single self-describing archive. RO-Crate is used in this protocol as the container format for the audit bundle: the package of log, attestation, and inclusion proof that a designer submits to a synthesis provider or regulator (Leo et al., 2024).

root hash: A single hash value that summarizes an entire hash chain or Merkle tree. Computing the root hash of a log allows a verifier to confirm the integrity of the entire log by checking a single value rather than rehashing every individual record (Laurie, 2014).

sequence similarity: A measure of how closely two biological sequences resemble each other. High sequence similarity to a known pathogen or toxin is used as a risk indicator in synthesis screening. The protocol's domain policy mechanism allows a notary to check whether any output sequence exceeds a defined similarity threshold before issuing a signed attestation (Altschul et al., 1990).

Sigstore and Rekor: Sigstore is an open-source project providing free infrastructure for signing, verifying, and recording software artifacts. Rekor is Sigstore’s transparency log component: a public, append-only ledger that records signed attestations and provides tamper-evident time stamping. In this protocol, Rekor (or a compatible transparency log) serves as the public anchor that makes backdating of audit records detectable (Newman, Meyers, and Torres-Arias, 2022).

Supply-Chain Levels for Software Artifacts (SLSA): A security framework developed by Google and the Open Source Security Foundation that defines graduated levels of assurance for software build and release processes. SLSA provenance attestations, which describe how a software artifact was built, serve as a model for the design run provenance attestations in this protocol (OpenSSF Supply Chain Integrity Working Group, undated).

transparency log: A public, append-only ledger in which signed records are stored in a Merkle tree structure. Because records can only be added and never deleted or modified and because the log’s state is periodically published and auditable by anyone, transparency logs make it possible to detect backdating and provide publicly verifiable proof of when a particular record was submitted. Certificate transparency logs, which record the issuance of website security certificates, are the most widely deployed example (Laurie, 2014).

trusted execution environment (TEE): A hardware-isolated region within a processor, such as Intel SGX or Advanced Micro Devices Secure Encrypted Virtualization, that executes code and stores data in a way that is protected from the rest of the system, including the operating system and the device owner. Code running inside a TEE can be remotely attested, meaning that an outside party can verify what software is running there. In this protocol, the recorder runs inside a TEE so that the designer cannot access or forge the signing key, even on their own machine (Chakrabarti et al., 2020).

trusted platform module (TPM): On-board security chip that provides sealed key storage, random-number generation, and remote measurement and attestation of firmware and software state (Trusted Computing Group, 2019).

World Wide Web Consortium PROV Ontology (W3C PROV-O): A standard vocabulary, published by W3C, for describing provenance—that is, the origin and history of data and computational processes. PROV-O defines such concepts as *activity* (a computational step), *entity* (a data artifact), and *agent* (a person or system responsible for an activity). In this protocol, PROV-O triples form the base vocabulary of the structured provenance envelope, ensuring that log records have a standardized, interoperable structure (Lebo, Sahoo, and McGuinness, 2013).

X.509 certificate: A standardized digital document that binds a public key to an identity, such as a person, organization, or software process, and is signed by a trusted certificate authority. X.509 certificates are the foundation of trust on the web (used in HTTPS) and in enterprise identity management. In this protocol, short-lived X.509 certificates are issued to the TEE to authorize signing operations during a design run (Ceccanti, Vianello, and Giacomini, 2020).

References

- Alterovitz, Gil, Dennis Dean, Carole Goble, Michael R. Crusoe, Stian Soiland-Reyes, Amanda Bell, Anais Hayes, Anita Suresh, Anjan Purkayastha, Charles H. King, et al., “Enabling Precision Medicine via Standard Communication of HTS Provenance, Analysis, and Results,” *PLoS Biology*, Vol. 16, No. 12, December 2018.
- Altschul, Stephen F., Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman, “Basic Local Alignment Search Tool,” *Journal of Molecular Biology*, Vol. 215, No. 3, October 1990.
- Appel, Andrew W., “Verification of a Cryptographic Primitive: SHA-256,” *ACM Transactions on Programming Languages and Systems*, Vol. 37, No. 2, April 2015.
- Aref, Yazan, Zakiya Alfughi, and Abdelkader Ouda, “Evaluating Hardware Security Modules: Key Factors for Choosing the Best Option for Organizational Security Requirements,” *2025 International Conference on Smart Applications, Communications and Networking (SmartNets)*, July 2025.
- Baek, Minkyung, Frank DiMaio, Ivan Anishchenko, Justas Dauparas, Sergey Ovchinnikov, Gyu Rie Lee, Jue Wang, Qian Cong, Lisa N. Kinch, R. Dustin Schaeffer, et al., “Accurate Prediction of Protein Structures and Interactions Using a Three-Track Neural Network,” *Science*, Vol. 373, No. 6557, August 20, 2021.
- Bennett, Nathaniel R., Brian Coventry, Inna Goreschnik, Buwei Huang, Aza Allen, Dionne Vafeados, Ying Po Peng, Justas Dauparas, Minkyung Baek, Lance Stewart, et al., “Improving De Novo Protein Binder Design with Deep Learning,” *Nature Communications*, Vol. 14, No. 1, 2023.
- Buckmann, Annalina, Jan Magnus Nold, Yasemin Acar, and Yixin Zou, “More Than Usability: Differential Access to Digital Security and Privacy,” *Proceedings of the Twenty-First Symposium on Usable Privacy and Security (SOUPS 2025)*, August 2025.
- Burland, Timothy G., “DNASTAR’s Lasergene Sequence Analysis Software,” in S. Misener and S. A. Krawetz, eds., *Methods in Molecular Biology*: Vol. 132, *Bioinformatics Methods and Protocols*, Humana, 2000.
- Ceccanti, Andrea, Enrico Vianello, and Francesco Giacomini, “Beyond X.509: Token-Based Authentication and Authorization in Practice,” *24th International Conference on Computing in High Energy and Nuclear Physics (CHEP 2019)*, Vol. 245, 2020.
- Chakrabarti, Somnath, Thomas Knauth, Dmitrii Kuvaiskii, Michael Steiner, and Mona Vij, “Trusted Execution Environment with Intel SGX,” in Xiaoqian Jiang and Haixu Tang, eds., *Responsible Genomic Data Sharing: Challenges and Approaches*, Academic Press, 2020.
- Chhetri, Tek Raj, Yaroslav O. Halchenko, Dorota Jarecka, Puja Trivedi, Satrajit S. Ghosh, Patrick Ray, and Lydia Ng, “Bridging the Scientific Knowledge Gap and Reproducibility: A Survey of Provenance, Assertion and Evidence Ontologies,” *WWW ’25: Companion Proceedings of the ACM Web Conference 2025*, May 2025.
- Crawford, Forrest W., Kyle Webster, Gerald L. Epstein, Derek Roberts, Joseph Fair, and Sella Nevo, “Securing Commercial Nucleic Acid Synthesis,” *RAND Health Quarterly*, Vol. 12, No. 1, 2024.
- Daum, Berthold, *Modeling Business Objects with XML Schema*, Morgan Kaufmann Publishers and dpunkt.verlag, 2003.
- Dauparas, J., I. Anishchenko, N. Bennett, H. Bai, R. J. Ragotte, L. F. Milles, B. I. M. Wicky, A. Courbet, R. J. de Haas, N. Bethel, et al., “Robust Deep Learning–Based Protein Sequence Design Using ProteinMPNN,” *Science*, Vol. 378, No. 6615, September 15, 2022.
- Felber, Pascal, “Big Data Processing: Security and Scalability Challenges,” *24th International Conference on Principles of Distributed Systems (OPODIS 2020)*, Vol. 184, 2021.
- Fontana, Pietro, Ying Dong, Xiong Pi, Alexander B. Tong, Corey W. Hecksel, Longfei Wang, Tian-Min Fu, Carlos Bustamante, and Hao Wu, “Structure of Cytoplasmic Ring of Nuclear Pore Complex by Integrative Cryo-EM and AlphaFold,” *Science*, Vol. 376, No. 6598, June 10, 2022.
- Go, “Sumdb,” software package, version 0.34.0, March 10, 2026. As of May 21, 2026: <https://pkg.go.dev/golang.org/x/mod/sumdb>

Godbold, Gene D., Anthony D. Kappell, Danielle S. LeSassier, Todd J. Treangen, and Krista L. Ternus, “Categorizing Sequences of Concern by Function to Better Assess Mechanisms of Microbial Pathogenesis,” *Infection and Immunity*, Vol. 90, No. 5, May 2022.

Godbold, Gene D., and Matthew B. Scholz, “Annotation of Functions of Sequences of Concern and Its Relevance to the New Biosecurity Regulatory Framework in the United States,” *Applied Biosafety*, Vol. 29, No. 3, September 2024.

Godbold, Gene D., Krista L. Ternus, Kevin Flyangolts, Nicole Wheeler, Michael Parker, Jacob Beal, Peter A. Carr, Kemper Talley, Caitlin Jagla, Bryan T. Gemler, et al., “The Case for Limiting ‘Sequences of Concern’ to Those with Demonstrated Pathogenic Function,” *Applied Biosafety*, Vol. 30, No. 3, September 2025.

GoogleContainerTools/Distroless Maintainers, “Distroless’ Container Images,” GitHub, undated. As of May 26, 2026:

<https://github.com/googlecontainertools/distroless>

Grunert, Sascha, “Verifying Container Image Signatures Within CRI Runtimes,” *Kubernetes Blog*, June 29, 2023. As of May 15, 2026:

<https://kubernetes.io/blog/2023/06/29/container-image-signature-verification>

Hardwicke, Tom E., Maya B. Mathur, Kyle MacDonald, Gustav Nilsson, George C. Banks, Mallory C. Kidwell, Alicia Hofelich Mohr, Elizabeth Clayton, Erica J. Yoon, Michael Henry Tessler, et al., “Data Availability, Reusability, and Analytic Reproducibility: Evaluating the Impact of a Mandatory Open Data Policy at the Journal *Cognition*,” *Royal Society Open Science*, Vol. 5, No. 8, August 2018.

Hayes, Jane Huffman, Jared Payne, Emily Essex, Kelsey Cole, Joseph Alverson, Alex Dekhtyar, Dongfeng Fang, and Grant Bernosky, “Towards Improved Network Security Requirements and Policy: Domain-Specific Completeness Analysis via Topic Modeling,” *2020 IEEE Seventh International Workshop on Artificial Intelligence for Requirements Engineering (AIRE)*, September 2020.

Imperiale, Michael J., and Arturo Casadevall, “A New Synthesis for Dual Use Research of Concern,” *PLoS Medicine*, Vol. 12, No. 4, April 2015.

Intel Corporation, *Intel® Software Guard Extensions (Intel® SGX) Developer Guide*, 2022.

James, Jose K., Kristjan Norland, Angad S. Johar, and Iftikhar J. Kullo, “Deep Generative Models of LDLR Protein Structure to Predict Variant Pathogenicity,” *Journal of Lipid Research*, Vol. 64, No. 12, December 2023.

Jeliazkov, Jeliazko R., Diego del Alamo, and Joel D. Karpik, “ESMFold Hallucinates Native-Like Protein Sequences,” *bioRxiv*, May 24, 2023.

Jumper, John, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al., “Highly Accurate Protein Structure Prediction with AlphaFold,” *Nature*, Vol. 596, July 15, 2021a.

Jumper, John, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al., “Applying and Improving AlphaFold at CASP14,” *Proteins*, Vol. 89, No. 12, December 2021b.

Kidwell, Mallory C., Ljiljana B. Lazarević, Erica Baranski, Tom E. Hardwick, Sarah Piechowski, Lina-Sophia Falkenberg, Curtis Kennett, Agnieszka Slowik, Carina Sonnleitner, Chelsey Hess-Holden, et al., “Badges to Acknowledge Open Practices: A Simple, Low-Cost, Effective Method for Increasing Transparency,” *PLoS Biology*, Vol. 14, No. 5, May 2016.

Laurie, Ben, “Certificate Transparency,” *Communications of the ACM*, Vol. 57, No. 10, October 2014.

Lebo, Timothy, Satya Sahoo, and Deborah McGuinness, eds., “PROV-O: The PROV Ontology,” World Wide Web Consortium, 2013.

Lefevre, Charlotte, and Bart Mennink, “Permutation-Based Hash Chains with Application to Password Hashing,” *IACR Transactions on Symmetric Cryptology*, Vol. 2024, No. 4, 2024.

Leo, Simone, Michael R. Crusoe, Laura Rodríguez-Navas, Raúl Sirvent, Alexander Kanitz, Paul De Geest, Rudolf Wittner, Luca Pireddu, Daniel Garijo, José M. Fernández, et al., “Recording Provenance of Workflow Runs with RO-Crate,” *PLoS ONE*, Vol. 19, No. 9, September 10, 2024.

- Lu, Amy X., Jason Yang, Yueming Long, Theophile Lambert, Wilson Yan, Sarah A. Robinson, Kevin K. Yang, Vladimir Gligorijevic, Kyunghyun Cho, Richard Bonneau, et al., “Controllable All-Atom Protein Generation with Latent Diffusion,” *bioRxiv*, last updated January 7, 2026.
- Manfredi, Matteo, Castrense Savojardo, Georgii Iardukhin, Davide Salomoni, Alessandro Costantini, Pier Luigi Martelli, and Rita Casadio, “Alpha&ESMhFolds: A Web Server for Comparing AlphaFold2 and ESMFold Models of the Human Reference Proteome,” *Journal of Molecular Biology*, Vol. 436, No. 17, September 2024.
- Ménétrey, Jâmes, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, Giovanni Mazzeo, Arne Hollum, and Darshan Vaydia, “A Comprehensive Trusted Runtime for WebAssembly with Intel SGX,” *IEEE Transactions on Dependable and Secure Computing*, Vol. 21, No. 4, July–August 2024.
- Merkle, Ralph C., “A Certified Digital Signature,” *Advances in Cryptology—CRYPTO ’89 Proceedings*, 1989.
- Mohanty, Mohan Debarchan, Abhishek Das, Mihir Narayan Mohanty, Ayman Altameem, Soumya Ranjan Nayak, Abdul Khader Jilani Saudagar, and Ramesh Chandra Poonia, “Design of Smart and Secured Healthcare Service Using Deep Learning with Modified SHA-256 Algorithm,” *Healthcare*, Vol. 10, No. 7, July 2022.
- National Science Advisory Board for Biosecurity, *Proposed Biosecurity Oversight Framework for the Future of Science*, National Institutes of Health, Office of Science Policy, March 2023.
- Newman, Zachary, John Speed Meyers, and Santiago Torres-Arias, “Sigstore: Software Signing for Everybody,” *CCS ’22: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, Association for Computing Machinery, November 2022.
- OpenSSF Supply Chain Integrity Working Group, “SLSA Specification,” webpage, undated. As of May 15, 2026: <https://slsa.dev/spec/v1.2>
- Pearson, William R., and David J. Lipman, “Improved Tools for Biological Sequence Comparison,” *Proceedings of the National Academy of Sciences of the United States of America*, Vol. 85, No. 8, April 1988.
- Poniszewska-Marañda, Aneta, and Ewa Czechowska, “Kubernetes Cluster for Automating Software Production Environment,” *Sensors (Basel)*, Vol. 21, No. 5, March 2021.
- Reproducible Builds, homepage, undated. As of May 21, 2026: <https://reproducible-builds.org>
- Simonyan, Vahan, Jeremy Goecks, and Raja Mazumder, “Biocompute Objects—a Step Towards Evaluation and Validation of Biomedical Scientific Computations,” *PDA Journal of Pharmaceutical Science and Technology*, Vol. 71, No. 2, March 2017.
- Smith, Jacob, “Daily Briefing: Google DeepMind Roll Out ‘Watermark’ on AI-Generated Text,” *Nature*, October 24, 2024.
- Tani, Jowy, “Biodesign in the Generative AI Era: Enhancing Innovation and Equity with NLP and LLM Tools,” *BMJ Health and Care Informatics*, Vol. 32, No. 1, February 5, 2025.
- Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos, “In-Toto: Providing Farm-to-Table Guarantees for Bits and Bytes,” *USENIX Security Symposium 2019*, August 2019.
- Trusted Computing Group, “Trusted Platform Module 2.0 Library Specification, Revision 1.59,” November 2019.
- Wang, Jie, and Juan Wang, “The Design and Implementation of a Secure and Efficient Firmware Trusted Platform Module for RISC-V Platforms,” *Journal of Electronics and Information Technology*, Vol. 47, No. 7, July 2025.
- Wang, Mengdi, Zaixi Zhang, Amrit Singh Bedi, Alvaro Velasquez, Stephanie Guerra, Sheng Lin-Gibson, Le Cong, Yuanhao Qu, Souradip Chakraborty, Megan Blewett, et al., “A Call for Built-In Biosecurity Safeguards for Generative AI Tools,” *Nature Biotechnology*, Vol. 43, No. 6, June 2025.
- Wang, Y., and D. Simmons, “Enhanced Enterprise Web-Based Application Security Using GeoIP Services,” *Proceedings of the 10th IASTED International Conference on Software Engineering and Applications*, January 2006.
- Watson, Joseph L., David Juergens, Nathaniel R. Bennett, Brian L. Trippe, Jason Yim, Helen E. Eisenach, Woody Ahern, Andrew J. Borst, Robert J. Ragotte, Lukas F. Milles, et al., “De Novo Design of Protein Structure and Function with RFdiffusion,” *Nature*, Vol. 620, No. 7976, August 2023.

Weber, Sam, Michael Coblenz, Brad Myers, Jonathan Aldrich, and Joshua Sunshine, “Empirical Studies on the Security and Usability Impact of Immutability,” *2017 IEEE Cybersecurity Development (SecDev)*, September 2017.

World Health Organization, *Global Guidance Framework for the Responsible Use of the Life Sciences: Mitigating Biorisks and Governing Dual-Use Research*, 2022.

Yao, Maoqun, Ziwei Xue, Haiwei Li, and Shanhu Shen, “An Optimized Hardware Implementation of SHA-256 Round Computation,” *Computer Journal*, Vol. 68, No. 4, April 2025.

Zalewski, Mateusz, Björn Wallner, and Sebastian Kmiecik, “Protein-Peptide Docking with ESMFold Language Model,” *Journal of Chemical Theory and Computation*, Vol. 21, No. 6, March 25, 2025.

Zhang, Zaixi, Ruofan Jin, Guangxue Xu, Xiaotong Wang, Marinka Zitnik, Le Cong, and Mengdi Wang, “FoldMark: Safeguarding Protein Structure Generative Models with Distributional and Evolutionary Watermarking,” *bioRxiv*, June 4, 2025.

About the Authors

Allison Berke is a senior engineer at the Meselson Center at RAND. She conducts technical and policy research on biosecurity and biosafety. She holds a Ph.D. in bioengineering.

Kyle A. Kilian is an AI and information security analyst at RAND. His research is focused on global, over-the-horizon risks from emerging technologies. He holds graduate degrees in data science and cyber intelligence and international affairs.

Graham Griffin is a Ph.D. student at the RAND School of Public Policy and an assistant policy researcher at RAND. His research is on science and technology policy, particularly balancing innovation with technological risk. He has an M.S. in chemistry.

Jay Atanda is a senior policy researcher at the Meselson Center at RAND. His research examines development of new technological tools, the potential to misuse them, and their convergence with traditional threats. He holds a D.M.D., an M.P.H., and a Dr.P.H.

Enrique Vazquez is a former research assistant at RAND. His work includes research and policy analysis investigating the impact of health policy decisions on accessibility and care delivery. He holds dual B.A. degrees in molecular biology and political science.